



Keystroke Dynamics: Concepts, Techniques, and Applications

RASHIK SHADMAN, Electrical and Computer Engineering, Clarkson University, Potsdam, United States

AHMED ANU WAHAB, Clarkson University, Potsdam, United States

MICHAEL MANNO, Air Force Research Laboratory Information Directorate, Rome, United States
and Electrical and Computer Engineering, Clarkson University, Potsdam, USA

MATTHEW LUKASZEWSKI, Clarkson University, Potsdam, United States

DAQING HOU, Electrical and Computer Engineering, Clarkson University, Potsdam, United States
and Software Engineering, Rochester Institute of Technology, Rochester, USA

FARAZ HUSSAIN, Electrical and Computer Engineering, Clarkson University, Potsdam, United States

Reliably identifying and verifying subjects remains integral to computer system security. Various novel authentication techniques, such as biometric authentication systems, have been developed in recent years. This article provides a detailed review of keystroke-based authentication systems and their applications. Keystroke dynamics is a behavioral biometric that is emerging as an important tool for cybersecurity as it promises to be nonintrusive and cost-effective. In addition, no additional hardware is required, making it convenient to deploy. This survey covers novel keystroke datasets, state-of-the-art keystroke authentication algorithms, keystroke authentication on touch screen and mobile devices, and various prominent applications of such techniques beyond authentication. The article covers all the significant aspects of keystroke dynamics and can be considered a reference for future researchers in this domain.

The article includes a discussion of the latest keystroke datasets, providing researchers with an up-to-date resource for analysis and experimentation. In addition, this survey covers the state-of-the-art algorithms adopted within this domain, offering insights into the cutting-edge techniques utilized for keystroke analysis. Moreover, this article explains the diverse applications of keystroke dynamics, particularly focusing on security, verification, and identification uses. Beyond these crucial areas, we mention additional applications where keystroke dynamics can be applied, broadening the scope of understanding regarding its potential impact across various domains. Unlike previous survey articles, which typically concentrate on specific aspects of keystroke dynamics, our comprehensive analysis presents all relevant areas within this field. By introducing discussions on the latest advances, we provide readers with a thorough understanding of the current landscape and emerging trends in keystroke dynamics research. Furthermore, this article presents

This material is based upon work supported by the Center for Identification Technology Research and the National Science Foundation under Grant No. 2413228 as well as 2122746 and 2526924.

Authors' Contact Information: Rashik Shadman, Electrical and Computer Engineering, Clarkson University, Potsdam, New York, United States; e-mail: shadmar@clarkson.edu; Ahmed Anu Wahab, Clarkson University, Potsdam, New York, United States; e-mail: wahabaa@clarkson.edu; Michael Manno, Air Force Research Laboratory Information Directorate, Rome, New York, United States and Electrical and Computer Engineering, Clarkson University, Potsdam, New York, USA; e-mail: michael.manno@us.af.mil; Matthew Lukaszewski, Clarkson University, Potsdam, New York, United States; e-mail: lukaszmp@clarkson.edu; Daqing Hou, Electrical and Computer Engineering, Clarkson University, Potsdam, New York, United States and Software Engineering, Rochester Institute of Technology, Rochester, New York, USA; e-mail: dhoul@clarkson.edu; Faraz Hussain, Electrical and Computer Engineering, Clarkson University, Potsdam, New York, United States; e-mail: fhussain@clarkson.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/06-ART283

<https://doi.org/10.1145/3733103>

a summary of future research opportunities, highlighting potential areas for exploration and development within the realm of keystroke dynamics. This forward-looking perspective aims to inspire further inquiry and innovation, guiding the trajectory of future studies in this dynamic field.

CCS Concepts: • **Security and privacy** → **Biometrics**; • **Computing methodologies** → **Machine learning**;

Additional Key Words and Phrases: keystroke dynamics, behavioral biometrics, authentication, identification, cybersecurity

ACM Reference Format:

Rashik Shadman, Ahmed Anu Wahab, Michael Manno, Matthew Lukaszewski, Daqing Hou, and Faraz Hus-sain. 2025. Keystroke Dynamics: Concepts, Techniques, and Applications. *ACM Comput. Surv.* 57, 11, Article 283 (June 2025), 35 pages. <https://doi.org/10.1145/3733103>

1 Introduction

User authentication is a process that verifies that someone who is attempting to access services and applications is who they claim to be. There are three broad categories of approaches to authenticating a user. The first category is based on *What you know*, such as the username and password or answers to security questions. This is also known as **Knowledge-Based Authentication (KBA)**. The second category is based on *What you have*, i.e., a physical object that can be possessed, for example, a security badge or a security key such as Yubikey. The third category is based on *Who you are*, which can be (i) *physiological*, e.g., face, fingerprint, or iris, (ii) *behavioral*, e.g., keystroke dynamics, mouse dynamics, mobile motion or swipes. Although KBA is currently still the dominant approach for identity management, there are active efforts in creating alternatives such as face recognition and FIDO.¹ In particular, an emerging field of user authentication is behavioral biometrics, which authenticates a user based on their behavior when interacting with a computing device.

Keystroke-based biometric authentication systems are a convenient and user-friendly authentication method in which a person is authenticated based on their typing pattern. Although such systems have great potential, their application in the real world so far has been limited. Keystroke biometric authentication systems have several advantages:

- (1) They are cost-effective as no additional hardware is required; only the regular keyboard is needed.

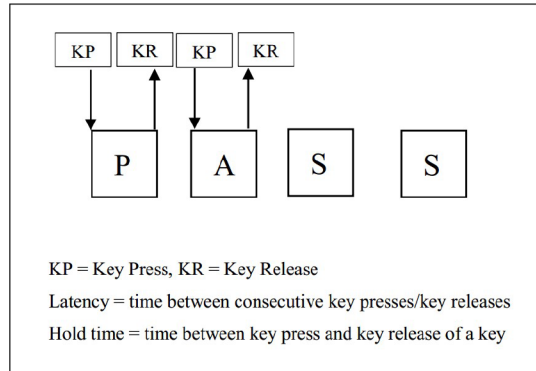


Fig. 1. Timing Features of Keystrokes. Latency and HT: these timing features are used to create a template of a user and at the time of testing these features are compared to verify the user. Between two consecutive keystrokes, there can be six timing features: HT of two keys, time between key press of first key and key press of second key = $KP2 - KP1$, time between key press of first key and key release of second key = $KR2 - KP1$, time between key release of first key and key press of second key = $KP2 - KR1$, time between key release of first key and key release of second key = $KR2 - KR1$.

¹FIDO stands for Fast IDentity Online. It's a set of open authentication standards designed to make online logins more secure and user-friendly by reducing reliance on passwords.

- (2) They are non-intrusive and do not create any extra hassle for the user.
- (3) The systems can be deployed remotely.

Keystroke authentication operates by creating a template for each user based on their typing pattern during an enrollment period. Once enrolled, the test sample from the user is contrasted with the representation of the same user and a matching score will be calculated. Matching scores are calculated on the basis of the timing features of keystrokes. Figure 1 provides an overview of the timing features of keystrokes.

Researchers use three types of keystroke data for authentication:

- (1) Free-text, also known as dynamic text, allows the user to type freely without any restriction [76]. For example, if a user writes a paragraph on a topic, it will be free-text. Free-text based keystroke dynamics can enable continuous authentication as a user types.
- (2) Fixed-text or static text [58]. Fixed-text is constant during the authentication process, during both template creation and testing. The keystroke data for a password can be used to create a second authentication factor.
- (3) Semi fixed-text shares some characteristics with free and fixed-text. Examples of semi fixed-text include Linux commands [116] and the webform keystroke data [115].

1.1 Justification for a New Survey

The oldest survey article we reviewed is Alsultan and Warwick's article from 2013 [11] that focused on free-text keystroke systems. Fixed-text or semi fixed-text keystroke systems were not included in their survey. In our survey, we focus on all three types of keystroke systems. Zhong and Deng [121] presented a comprehensive survey of keystroke dynamics; Giot, Dorizzi, and Rosenberger [38] provided a review of previously created keystroke benchmarking datasets; Ali et al. [9] presented an elaborate survey of the keystroke data collection details and classification algorithms. However, as these surveys were published in 2015, they do not cover recent trends in keystroke dynamics research or newer benchmark datasets, e.g., [22, 76, 104, 114, 115].

Saini, Kaur, and Bhatia [88] and Saifan et al. [87] presented a comprehensive review on keystroke dynamics research, especially for touch screen and mobile devices. However, these articles do not cover recent developments as they were published in 2016. Also, these articles focused only on mobile devices.

Shinde, Shetty, and Mehra [95] mainly focused on different aspects of static-text keystroke authentication. They did not discuss dynamic or continuous keystroke authentication in their survey. In our survey, we explain both static and dynamic keystroke authentication. Ali et al. [8] provided a comprehensive assessment of the newest datasets and algorithms used in keystroke dynamics research. This survey was conducted in 2017, and new datasets and algorithms are now available.

Sanghi and Arya [90] presented a review of the methods and metrics used in keystroke dynamics. This article described various applications of keystroke dynamics as well. Sadikan, Ramli, and Fudzee [85] performed a comprehensive analysis of the applications of keystroke dynamics. In this article, we describe the latest applications of keystroke dynamics. Tewari [108] discussed only deep learning algorithms for keystroke dynamics. This article provides a more comprehensive description of various types of keystroke dynamics algorithms.

To summarize, this survey covers keystroke dynamics performance metrics, the latest benchmark datasets for keystroke dynamics, state-of-the-art keystroke authentication algorithms, keystroke feature engineering techniques, keystroke dynamics for touch screen and mobile devices, and the most important applications of keystroke dynamics.

Table 1 shows a comparison of existing surveys on keystroke dynamics. We highlight their unique contributions, advantages, and disadvantages. We have categorized the studies based on

Table 1. A Summary of Recent Survey Articles Focusing on Keystroke Dynamics. The Surveys are Compared Across Five Categories Based on Whether They Cover Various Areas, viz. Datasets, Algorithms, Mobile Device Keystroke Capture, Feature Engineering Approaches, and Applications

Survey Article	Year	Citations	Covered Areas				
			Datasets	Algorithms	Mobile Devices	Feature Engineering	Applications
Alsultan and Warwick [11]	2013	115	Yes	Yes	No	No	Yes
Zhong and Deng [121]	2015	60	Yes	Yes	Yes	No	Yes
Giot, Dorizzi, and Rosenberger [38]	2015	43	Yes	No	No	No	No
Ali et al. [9]	2015	30	Yes	Yes	No	No	No
Saini, Kaur, and Bhatia [88]	2016	20	No	No	Yes	No	No
Saifan et al. [87]	2016	11	No	No	Yes	No	No
Shinde, Shetty and Mehra [95]	2016	7	Yes	Yes	No	No	No
Ali et al. [8]	2017	103	Yes	Yes	No	No	No
Sanghi and Arya [90]	2017	2	No	Yes	No	No	Yes
Sadikan, Ramli, and Fudzee [85]	2019	10	Yes	No	No	No	Yes
Maiorana, Kalita, and Campisi [68]	2021	4	No	No	Yes	No	No
Tewari [108]	2022	1	Yes	Yes	No	No	No
This Survey			Yes	Yes	Yes	Yes	Yes

their contribution to one of the following categories: keystroke datasets, keystroke authentication algorithms, keystroke feature engineering approaches, keystroke authentication on touch screen and mobile devices, and applications of keystroke dynamics.

1.2 Methodology

Through this article, we comprehensively cover recent developments in keystroke dynamics research and their applications. We collected articles from 2013–present on keystroke dynamics from sources such as Google Scholar, IEEE, and Academia.edu. The following search terms were used:

- keystroke dynamics.
- keystroke dataset/database.
- keystroke authentication algorithm.
- keystroke dynamics + mobile devices, keystroke dynamics + touch screen devices.

We categorized them based on their contribution and uniqueness. The following contributions were considered:

- Novel keystroke dataset/database.
- New methods to improve the authentication performance of keystroke dynamics-based systems such as novel/efficient keystroke authentication algorithm or software, effective keystroke feature engineering approach.
- Novel keystroke authentication technique based on touch screen and mobile devices.
- Potential application of keystroke dynamics.
- Future research problem/direction in the field of keystroke dynamics.

If an article contributed in any of the above areas, we have included it in this survey. Predominantly, articles published after 2013 were considered for this survey as we wanted to cover the developments of keystroke dynamics during the last decade. Previous surveys on keystroke dynamics focused on specific areas such as keystroke dynamics on touch screen and mobile devices, keystroke authentication algorithms, and keystroke datasets. An overview of the structure of this article is shown in Figure 2.

2 Keystroke Biometrics

Keystroke dynamics research has been ongoing for decades and the main intended application has been verification and identification of subjects. Modern computer systems contain vital user information making it important to ensure the security of computer systems. Authentication via

something a user knows (i.e., a password) is no longer sufficient to guarantee the security of sensitive data.

In the context of keystroke dynamics, authentication, verification, and identification have different goals. Authentication means ensuring that a user is who they claim to be by comparing their typing patterns with a previously stored profile. In order to confirm the access of the correct individual, authentication is used when logging into a system. Verification can be done during different stages of interaction. In order to detect unauthorized/imposter access continuously during a session, verification is performed. Identification seeks to determine the identity of an unknown user from a pool of candidates based on their typing patterns. In the identification process, the user input is effectively matched to the most similar profile within a database.

2.1 Security and Authentication

Sridhar, Vaidya, and Yawalkar [98] proposed a keystroke authentication system for intruder detection in secure virtual environments. For this purpose, authentication data from login attempts was stored. Feature sets were extracted from 200 samples of username and password data. Fuzzy Logic algorithms were used in order to develop signatures for each participant. The typing pattern of each participant was compared to the respective signature in order to authenticate the participants. Membership functions of fuzzy logic were implemented for authenticating a participant. For 200 samples, they achieved a FAR of 0.0% and a FRR of 0.0% for both Bell MF and Gaussian MF.

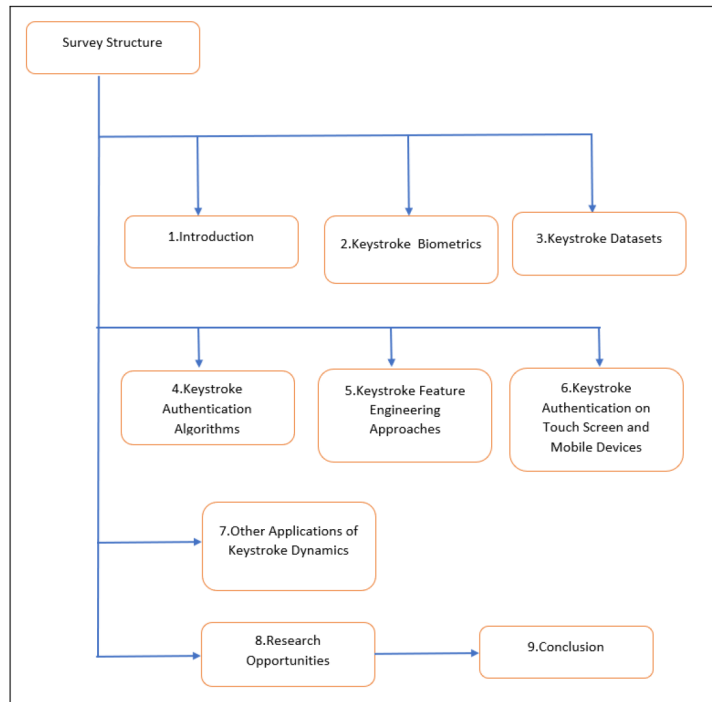


Fig. 2. An overview of the structure of this survey.

2.2 Person Identification and Forensic Investigation

Mondal and Bours [74] investigated the usefulness of keystroke dynamics in identifying a person. They proposed a **pairwise user coupling (PUC)** technique along with different machine learning algorithms. They also analyzed person identification via an optimized feature set. For their experiment, they used a novel keystroke dataset collected from three online exams. The subjects were PACE University undergraduates. 64 students participated in the data collection. The subjects typed a minimum of 500 keystrokes in each of the three exams. Mondal and Bours used a subset of data from the first exam as a training set. The remaining data was used for testing.

They implemented the PUC technique on their dataset using machine learning classifiers. The best accuracy achieved was 89.7%.

Mohlala, Ikuesan, and Venter [72] proposed a method based on keystroke dynamics to collect evidence for digital forensic readiness. The main goal was to collect solid keystroke evidence for forensic investigations. Their model consisted of two core phases: (1) the pattern development phase and (2) the pattern testing/validation phase. In the development phase, input data is taken from the known user and continually developed by applying the data preprocessing, machine learning, and pattern extraction subprocesses before storing the data in the database. They used an Android application with a software keyboard to record raw data. The dataset consisted of 42 subjects. The subjects typed fixed password with 51 samples per subject. For the experiment process, Mohlala, Ikuesan, and Venter implemented their developed tool on the benchmark dataset [13]. Each subject's pattern was accurately attributed to the respective user on the test set, except for one user. However, all subjects were accurately attributed using the attribution mechanism. Based on the results, it was clear that this study could achieve reliable accuracy for subject attribution in digital forensics.

2.3 User Authentication for Online Assessments and Examinations

Chen et al. [26] designed a keystroke dynamics-based user authentication system for online assessments and examinations. Their framework is based on an edge computing architecture. There were keystroke profiles of the users in the framework. In this study, Chen et al. used four features: **hold time (HT)**, **press flight time (FT)**, **HT proportion (HTP)**, and **FT proportion (FTP)**. There were two parts in the profile for static authentication and continuous authentication, respectively. In order to perform static keystroke authentication, they used an anomaly detector based on the Gaussian model. In the case of continuous keystroke authentication, keystroke events were collected continuously by the framework in order to form a continuous data stream of extracted keystroke timing features. For evaluation, three public datasets were used ([42, 58, 104]). To investigate the effectiveness of the framework, they used a real-world case of an online examination. In order to verify their basic detector, they used the dataset from Killourhy and Maxion [58] and achieved an EER of 6.62%. To evaluate the performance of static keystroke authentication, they used the dataset from Giot et al. [42] and achieved an EER of 5.71%. EER could be decreased to 4.03% using a dynamic user profiles mechanism. To evaluate the performance of continuous keystroke authentication, they used the dataset from Sun et al. [104] and achieved an average EER of 2%. The edge computing architecture was implemented to develop a prototype of an online examination system. There were three phases in the online examination system: registration phase, login phase, and testing phase. In order to simulate possible security-related situations, 27 users participated in the case study. The obtained results for this case study were promising.

2.4 Keystroke Dynamics Performance Metrics

There are a number of performance metrics used in keystroke dynamics, and each is used to present some specific information about the biometric system. This section describes all the metrics and when they are preferably used.

2.4.1 False Accept Rate (FAR) and False Reject Rate (FRR). The most common performance metrics used in biometric systems are the FAR and the FRR. The FAR is the measure of the likelihood that the biometric system will incorrectly grant access to an unauthorized user [33]. This is calculated as the ratio of the number of false acceptances divided by the total number of impostor attempts, as shown in Equation (1).

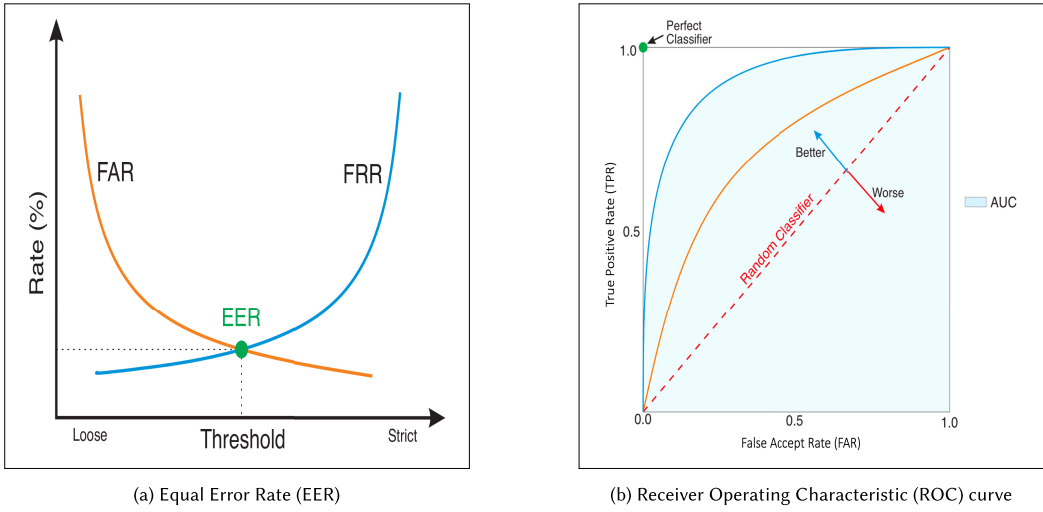


Fig. 3. (a) EER as the instance of FAR and FRR intersecting, and the effect of loose and strict threshold (when the threshold is loose, the FAR is high and FRR is low; when the threshold is strict, the FAR is low and FRR is high). The EER balances between loose and strict thresholds. (b) The ROC curve and the AUC show the performance of different classifiers. A classifier touching the point (0.0, 1.0) is desired and considered a perfect classifier. This is difficult in practical systems; a classifier closer to the coordinate (0.0, 1.0) is preferred. The ROC curve below the diagonal line is considered worse, while the ROC curve above the diagonal line is considered better.

The measure of the likelihood that an authorized user will be incorrectly rejected by the biometric system is called the FRR. This is measured by dividing the number of false rejections by the total number of genuine attempts, as shown in Equation (2).

The ideal value for both FAR and FRR is 0%, which translates to granting access to all genuine users while all impostors are caught. However, real-world biometric systems are not ideal. Therefore, for both FAR and FRR, a value as close to zero as possible is desired. A system with 0% FAR is considered to have the utmost security, while a system with 0% FRR is highly convenient. There is usually a tradeoff between security and convenience, as a highly secure system may reject too many genuine users, thus being inconvenient, while a highly convenient system may be insecure and therefore ineffective. The FAR is also known as the **False Positive Rate (FPR)**, while the $FRR = 1 - \text{True Positive Rate (TPR)}$.

$$FAR = \frac{\text{Number of false acceptances}}{\text{Total number of impostor attempts}}, \quad (1)$$

$$FRR = \frac{\text{Number of false rejections}}{\text{Total number of genuine attempts}}. \quad (2)$$

2.4.2 Equal Error Rate (EER). The EER is also a commonly used performance metric, especially when it is challenging to find an operating point between the FAR and FRR. The EER indicates the instance of FAR and FRR intersecting. Similar to the FAR and FRR, a lower EER value is preferred. As shown in Figure 3(a), a threshold closer to 0 favors convenience (low FRR) but is less secure (high FAR), while a threshold closer to 1 favors security (low FAR) but is less convenient (high FRR). The EER balances these two points.

2.4.3 Receiver Operating Characteristic (ROC) and Area Under the Curve (AUC). The ROC curve is a graphical plot of FAR and TPR (i.e., $1 - \text{FRR}$) that visualizes the performance of a binary classifier while the threshold for classification is changed. This metric provides a clear visualization of keystroke dynamics performance across varying classification thresholds. A perfect classifier will have a point at the top left corner with coordinates (0.0, 1.0) which illustrates no false accept or false reject (Figure 3(b)). Since this is hardly practical in a large-scale system, a classifier closer to the coordinate (0.0, 1.0) is desired. The diagonal from the top right to bottom left corners indicates a random classifier, which is similar to a random guess and, therefore, not desired. The AUC, often associated with the ROC curve, provides a combined performance measurement across all varying thresholds of classification.

2.4.4 Average Number of Impostor Actions (ANIA) and Average Number of Genuine Actions (ANGA). In keystroke dynamics, especially in continuous user authentication, performance metrics such as FAR, FRR, ROC, and AUC may not be sufficient, as they fail to illustrate the average keystroke count required for a genuine user to be rejected or an impostor to be detected. Therefore, to further evaluate the performance of a **continuous user authentication (CUA)** system, the ANIA and ANGA are used, where the actions in this case are keystrokes [75]. The ANGA is the number of legitimate actions users can perform before they are wrongfully denied access, while ANIA is the number of actions impostors can perform before they are detected and locked out. ANGA is preferred to be high (in fact, it is desired that a legitimate user should never be denied access), and ANIA as low as possible, so as to reduce the number of actions an illegitimate user can perform before detection.

The ANIA and ANGA are mostly used in keystroke dynamics continuous authentication and can be calculated as shown in Equations (3) and (4), where N is the number of consecutive actions a user (genuine or impostor) can finish before an authentication/identification decision is made.

$$\text{ANIA} = \frac{N}{1 - \text{FAR}}, \quad (3)$$

$$\text{ANGA} = \frac{N}{\text{FRR}}. \quad (4)$$

2.4.5 Authentication Time. The authentication time is an additional metric used in keystroke dynamics to describe the usability of the system. This metric is the total average time necessary for collecting keystroke data and the average of the time a system takes to permit or deny user access [113]. While the desire to achieve a very low FAR and FRR stands, it is important to achieve this with a very short authentication time. For example, the authentication system of Zheng et al. [120] achieved FAR and FRR of 1.30% but with a very long authentication time (approximately 38 minutes). Such a long authentication time would leave impostors uncaught, as most impostors require significantly less time to complete attacks. Additionally, the mouse dynamics-based authentication of Shen et al. [93] had 3.33% FAR and 2.12% FRR with about 119 seconds of authentication time. Although these FAR and FRR are higher than those of Zhen et al., it could be considered a better system based on the lower authentication time.

2.4.6 Rank-N. Rank-n [1] is an evaluation method used to measure the performance of keystroke dynamics-based authentication systems. A dataset of multiple samples from each subject is used to test the system. For each subject, one of the samples is used as a reference sample and the rest of the samples are considered as test samples. A similarity metric such as Euclidean distance is used to measure the similarity of a test sample to the reference sample. The test samples are ranked based on their similarity to the reference sample. The percentage of the inclusion of the correct

Table 2. A Summary of Publicly Available Keystroke Datasets. The Table Includes the Type of Data Collected, the Number of Subjects that Participated in the Data Collection, and the Average Number of Keystrokes Per Subject

Dataset	Type of data	Number of subjects	Average number of keystrokes per subject	Notes	Dataset and related software
CMU (2009) by Killourhy and Maxion [58]	Fixed	51	4400	Imposed password	http://www.cs.cmu.edu/~keystroke
GreyC-A (2009) by Giot, El-Abed, and Rosenberger [40]	Fixed	133	Unspecified	Subjects practiced on other keyboards before authenticating on the actual keyboard	http://www.ecole.ensicaen.fr/~rosenber/keystroke.html (currently unavailable) ²
GreyC-B (2012) by Giot, El-Abed, and Rosenberger [42]	Fixed	48	Unspecified	Imposed and individual credentials	http://www.epaymentbiometrics.ensicaen.fr/ (currently unavailable) ³
Pace (2013) by Bakelman et al. [17]	Fixed	30	220	Password and numeric input	Unknown
Clarkson I (2014) by Vural et al. [112]	Fixed and Free	39	21,533	Includes videos of subjects' facial expression and hand movements	Dataset available upon request from the authors
OhKBIC (2015) by Monaco et al. [73]	Free	64	minimum 1,500	One-handed	Unknown
Buffalo (2016) by Sun, Ceker, and Upadhyaya [104]	Fixed and Free	157	17,000	Desktop, and Multi-keyboard	http://cubs.buffalo.edu/research/datasets
Clarkson II (2017) by Murphy et al. [76]	Free	103	125,000	Desktop, and Completely uncontrolled	Dataset available upon request from the authors
Aalto Desktop (2018) by Dhakal et al. [32]	Free	168,000	800	Desktop	http://userinterfaces.aalto.fi/136Mkeystrokes
Aalto Mobile (2019) by Palin et al. [80]	Free	37,370	Unspecified	Mobile	https://userinterfaces.aalto.fi/typing37k
FACT (2020) by Kim and Kang [59]	Free	50	4000	Mobile and bilingual	https://doi.org/10.1016/j.patcog.2020.107556
AR (2021) by Wahab et al. [115]	Semi-Fixed	44	2,048	Account recovery	Dataset available upon request from the authors
Multi-K (2022) by Wahab et al. [114]	Free	86	24,000	Multi-keyboard and bilingual	Dataset available upon request from the authors

test sample within the top n-ranked samples is measured by the rank-n metric. If the correct test sample is included within the top five ranked samples 90% of the time, then the rank-5 accuracy is 90%. Therefore, the rank-n value needs to be lower for better performance. For a lower rank-n value, there is a better chance that the correct test sample will be ranked higher. A rank-1 accuracy of 90% indicates that the correct test sample is ranked 1st 90% of the time. Rank-n evaluation provides a comprehensive analysis of the system’s performance.

There are other performance metrics such as precision, recall, ACC (accuracy), BA (balanced accuracy), and the F1-score which are occasionally used in keystroke dynamics.

3 Keystroke Datasets

A keystroke dataset is required to perform a satisfactory evaluation of keystroke dynamics-based authentication algorithms. The quality, size, and nature (fixed, free, or semi-fixed text) of the dataset are crucial factors in evaluating the performance of these algorithms. To perform an evaluation, an algorithm is applied that compares the typing patterns of users based on keystroke features (see Figure 1). This information is used to authenticate users. This section details various keystroke datasets and their effectiveness in terms of authentication performance. A summary of the keystroke datasets is shown in Table 2.

3.1 CMU Dataset (2009)

Killourhy and Maxion [58] collected fixed text (password) keystroke data and evaluated 14 algorithms with that dataset. 51 subjects were included in the data collection. The fixed password was “*tie5RoanI*”. Each subject typed this password 400 times during eight sessions, with 50 attempts in each session. There was at least a one-day gap between successive sessions to reflect the temporal variation in typing of the subjects. A Microsoft Windows application was used for the data collection. An external reference clock was utilized to make more accurate timestamps. Killourhy and Maxion evaluated different algorithms with this dataset and achieved an EER between 9.6% and 10.2%.

3.2 GreyC-A Dataset (2009)

Giot, El-Abed, and Rosenberger [40] proposed a benchmark database and software for measuring the performance of authentication systems leveraging keystroke dynamics for user access control. Their focus was on static keystroke authentication (such as passwords or passphrases).

²Last attempted access: June 20, 2024.

³Last attempted access: June 20, 2024.

GREYC-Keystroke was the proposed software. They developed a standard keystroke dynamics database using the GREYC-Keystroke software. Keystroke data was collected for 133 participants including researchers, students, and employees. Each participant typed the password between 5 and 107 times. The authors collected the data of 7,555 attempts. Most participants attended at least five sessions. On average, there were 51 attempts per participant.

Giot et al. [41] used the GREYC keystroke dataset to evaluate their proposed technique based on SVM learning. The enrollment step was limited to five captures. The administrator set a passphrase that the users had to type. A two-class SVM was utilized for the enrollment. This SVM-based machine learning approach only needed five captured samples to develop the model. They used four types of algorithms for their experiment: (1) Statistical algorithm, (2) Distance-based algorithm, (3) Rhythm-based algorithm, and (4) Machine learning based algorithm. They also investigated the variations in the authentication method due to different keyboards. The authors also considered the consequences of some other factors like differing amounts of samples needed to develop the model, individual or global threshold, the database size, and so on. The EER was calculated by using ten vectors from the head of the dataset for enrollment and the remaining samples were used for verification. In terms of EER, their proposed method performed better than all other algorithms.

3.3 GreyC-B Dataset (2012)

Giot, El-Abed, and Rosenberger [42] created a new dataset for keystroke-based biometric systems. To build the dataset, two types of logins and passwords were chosen and imposed. Leveraging a web environment, the data was collected without any restrictions. They performed statistical analysis of some key factors, including the effect of password size on performance, the significance of fusion techniques, and so on. The data collection process consisted of three steps. (1) Ten attempts using a prior researcher-chosen login and password, (2) Ten attempts for the user-chosen login and password, and (3) Ten imposter samples from two other participants. 83 individuals participated in the data collection. They collected 5,185 genuine samples, 5,439 imposed samples, and 5,754 imposter samples. For each participant, 20 samples were used for training and the remaining samples were used for testing (with a minimum of 20). The conclusions of their experiment were: (1) Using an individual threshold was better than using a global threshold for computing the EER because an individual threshold is a value specific to a user that takes into account the origin of the sample i.e whether a legitimate user or an imposter, (2) Using logins provided better results than using passwords, (3) The performance improved by doing the fusion of all features (monographs and digraphs), (4) The performance was dependent on the password size and entropy.

3.4 Pace Dataset (2013)

Bakelman et al. [17] collected fixed-text numeric keypad data for keystroke authentication of 30 subjects over four days. On each day a maximum of 60 samples were collected from each subject. There were 11 keystrokes per sample from the numeric sequence and the *Enter* key. The data was collected by a third-party keylogger. The subjects had to type the samples with only their right hand in order to simulate typing a phone number or ATM PIN. There were 20 samples per subject. For this dataset, they got an EER of 6.1%.

3.5 Clarkson I Dataset (2014)

Vural et al. [112] developed a new dataset containing short pass-phrases, long text transcription (fixed-text) and free text. They also recorded videos showing subjects during data collection in order to capture the facial expressions and hand movements of the subjects. The dataset includes two kinds of fixed text (password and transcriptions) and free-text from 39 participants including university employees and students. For each participant, the data was collected in two one-hour

sessions each occurring on a different day. The first task was typing three different passwords. The subjects typed each password 20 times.

For the second task, the subjects had to answer nine survey questions in free-text with a minimum of 500 characters for each answer. The last task was copying the commencement speech of Steve Jobs at Stanford University. A browser-based keylogger was used for the data collection. The data was collected for a period of 11 months between August 2011 and June 2012 in a lab at Clarkson University. On average, each subject provided 11,066 keystrokes in session 1 and 10,467 keystrokes in session 2. Every session recording was analyzed to determine digraphs and trigraphs. N-graphs greater than 500 milliseconds or shorter than 30 milliseconds in length were discarded. In session 1, on average there were 8,406 digraphs and 389 unique digraphs per subject, 7,561 trigraphs, and 1,603 unique trigraphs per subject. In session 2, on average there were 8,261 digraphs and 388 unique digraphs per subject, 7,520 trigraphs, and 1,620 unique trigraphs per subject.

To evaluate this dataset, the algorithm of Leggett et al. [63] was used. A FAR of 0.25% and FRR of 17.65% were obtained for a threshold of 0.9; aFAR of 3.45% and FRR of 8.82% were obtained for a threshold of 0.85. Ceker and Upandhyaya [22] utilized this dataset to examine the accuracy of a single-class **support vector machine (SVM)** on longer text data, as opposed to shorter password text. They found that using the four ranking digraphs by commonality ('he', 're', 'th', and 'an') resulted in a 2.94% EER while using the most common 12 or more digraphs led to an almost 0% EER.

3.6 One-handed Keystroke Biometric Identification Competition (OhKBIC) Dataset (2015)

Monaco et al. [73] described the results of the OhKBIC. In the dataset, Monaco et al. included the data of normal typing and handicapped typing (typing with one hand). Data was collected from three online exams of undergraduate students. There were 64 subjects. In each exam, there were five essay questions. The subjects typed normally in the first exam. The subjects were directed to type with their left hand only for the second exam and with their right hand only for the third exam. Each subject typed a minimum of 500 keystrokes on each exam. The data was collected by a JavaScript-based keylogger and transferred to a server. Each used different keyboard make and models throughout the three exams which was a drawback of this dataset. A portion of the data collected from the first exam of normal both-hands typing was used as labeled training data. The unlabeled data included all three types of data (both-hand) typing, left-hand typing, and right-hand typing data. The top team used two regression models (Artificial Neural Network and Counter-Propagation Artificial Neural Network) and one prediction model (SVM) for pairwise coupling [48]. The best recognition rate ranged from 55.7% to 82.8% for both-hand sections, 19.9% to 30.5% for the left-hand section, and 24.8% to 34.3% for the right-hand section.

3.7 Buffalo Dataset (2016)

Sun, Ceker, and Upadhyaya [104] developed a shared dataset containing keystrokes collected from 157 participants. The data was both fixed and free-text. The data collection process continued for four months between September and December 2015. For each subject, the data was collected in three sessions. The tasks included fixed-text transcription and answering questions in free text. On average there was a 28 days gap between two sessions to consider the temporal variations. There are two sections in the dataset- in one section the subjects used the same keyboard across sessions. In the other section, the subjects used different keyboards across sessions. The first task was copying the 2005 commencement speech of Steve Jobs at Stanford University. The speech was divided into three parts of equal length; the subjects had to type one part in each session. The second task consisted of multiple subtasks including answering survey questions and describing a picture,

sending an e-mail with an attachment, and free internet browsing. Each session lasted for about 50 minutes with 30 minutes for the first task and 20 minutes for the second task. A system logger was adopted for the data collection on the Windows platform [37][105]. Among the 157 subjects, the data of 148 subjects was included in the dataset. The data of the remaining users was discarded due to mistakes. On average each subject typed about 5,700 keystrokes in each session, around 17,000 keystrokes for all three sessions. There was a 3 to 5 week time gap between successive sessions to reflect the temporal effect. The **Gaussian mixture model (GMM)**, a probabilistic model for classifying data into different categories based on the probability distribution, was utilized to evaluate this dataset. The EER was 0.01% and 0.39% for the GMM with one and two components, respectively.

3.8 Clarkson II Dataset (2017)

A novel dataset was proposed by Murphy et al. [76] that included keystrokes, mouse events, and active programs. 103 users contributed the data for a period of around 2.5 years. Unlike the other datasets, the data was collected on the users' personal computers during normal use. Keystrokes and mouse data were recorded while users make use of their computers, such as for typing, gaming, and so on, thereby classifying the data as free-text. The users provided 12.9 M keystrokes, with each user providing an average of 125 K keystrokes. A keylogger was installed on each user's computer to record keystroke and mouse movement events. The recorded data was sent to a remote database server. 10 samples, each sample consisting of 1,000 keystrokes, were used to form the reference profile of a user. The rest of the user's data was considered genuine test samples. For each user, the data of all other users was considered as imposter data. The EER was 10.36% for this dataset. This result was worse compared to fixed-text and controlled free-text datasets because it is a completely uncontrolled dataset and as such contains several keystrokes that are not usually seen in typing tasks.

3.9 Aalto Datasets (2018; 2019)

The Aalto University desktop [32] and mobile [80] datasets are large-scale datasets collected using an online typing test on desktop computers and mobile devices. The desktop dataset has 136 million keystrokes gathered over three months from 168,000 subjects. Participants were instructed to transcribe fifteen sentences in English by typing them as fast and accurately as possible. The 15 sentences were randomly taken from a set of 1,525 samples consisting of at least three words and a maximum of 70 characters. Subjects were allowed to make typing errors, correct them, or add new characters when typing, and as a result, they could type more than 70 characters. The dataset is categorized as a controlled free-text dataset because it involves transcribing, subjects did not type contents of their own but were shown what to type. There were 15 sessions and participants transcribed only one sentence during each session. The front end of the online test was deployed as an HTML web page leveraging CSS for website styling. JavaScript was used for dynamic events on the website such as keypresses and the data were stored in a MySQL database.

The Aalto mobile dataset [80] was collected from 37,370 participants performing transcription services on a website. It is an extension of the Aalto desktop dataset and follows the same procedure but on mobile devices. Subjects were to memorize a given sentence, and then type it quickly and accurately. Being a web-based method with a browser-side logger, as opposed to laboratory-based, there was less control over the quality of the data collected, and web applications on mobile devices have limited access privileges. As a result, many participants' data were found with undefined keycodes; many devices' down and up keystroke events were equal, producing invalid keystroke duration of <10 ms; and the keycodes of some pressed keys were missing.

3.10 AR Dataset (2021)

Wahab et al. [115] collected a new dataset of 500,000 keystrokes, designed for account recovery, to determine the performance of five differing keystroke dynamics-based algorithms [46, 53, 57, 58]: Euclidean distance, Manhattan distance, Scaled Manhattan distance, Mahalanobis distance, and the Gunetti and Picardi's algorithm. A total of 44 subjects were used during data collection including both students and university employees. The subjects had to complete an account recovery form that contained different fields. This type of data is called semi-fixed as it is neither free-text nor fixed-text. Two collection sessions were hosted to gather the data. In the first session, the subjects had to fill out an enrollment form ten times. The data collected from session one was used to create the profile of a subject. In the second session, the subjects had to fill out the same form again five times. The data collected from session two was used as genuine keystroke data for the corresponding subject and imposter data for other subjects. There was a one or two-week gap between the sessions. The fields of the enrollment form were—*Full name, Address, City, Zip, Phone, E-mail, Declaration, Password*. 28 subjects attended the second session and 16 subjects acted as imposter. Five algorithms were used for the evaluation of this account recovery dataset. The Scaled Manhattan distance algorithm showed the best performance. The best EER was 5.47% for individual fields, 0% for five fields combined, and 0% for seven fields combined.

3.11 Multi-K Dataset (2022)

Wahab et al. [114] created two novel free-text keystroke dynamics datasets. The first was a multi-keyboard keystroke dataset. Data was collected by leveraging four physical keyboards: mechanical, membrane, ergonomic, and laptop keyboards. For the second dataset, data was collected in both English and Chinese languages. 86 users participated in the data collection: 60 users for the multi-keyboard dataset and 26 users for the bilingual dataset. A keylogger deployed on a website was leveraged for the collection of both datasets.

To examine the impact of multi-keyboard and multi-language data in keystroke dynamics, Wahab et al. employed two free-text algorithms: the **Instance-based Tail Area Density (ITAD)** metric and the D-Vectors model. On average, each user provided around 14,000 keystrokes for the multi-keyboard dataset. For each user, per keyboard type, data from the initial visit was used for enrollment, while data from the second visit was used for testing. There were 200 keystrokes in each test sample, and the performance was measured using the EER. The results indicated that enrolling and testing with different keyboards affect keystroke dynamics performance where keyboard size and layout were two significant factors.

In the bilingual experiment, their aim was to investigate the impact of multi-language data in keystroke dynamics. On average, each user provided around 10,000 keystrokes for both English and Chinese languages. Monograph and digraph features were extracted from the dataset. Data from the first two questions was used for enrollment and data from the other questions was used for testing, with around 900 keystrokes used for enrollment and each test sample consisting of 200 keystrokes. The results revealed a performance degradation for cross-language in keystroke dynamics, with a performance loss of 14% when enrollment was done with Chinese data and testing was done with English data, and a loss of 6.4% when enrollment was done with English data and testing was done with Chinese data.

3.12 Touch Screen and Mobile Datasets

El-Abed, Dafer, and El Khayat [34] developed a new benchmark named RHU Keystroke for touch screen keystroke dynamics. They used a Windows Phone app for data collection. The data was stored in a keystroke dynamics database and necessary features were extracted. The participants

in the data collection were students of varying backgrounds to ensure the scenario was as real as possible. A total of 53 participants participated in three sessions typing the password *rhu.university* 15 times in each session. On average there was a five days gap between successive sessions. The RHU Keystroke benchmark consists of four timing features: PP, PR, RP, and RR.

Stragapede et al. [101] created a new database of mobile **Human-Computer Interaction (HCI)** named BehavePassDB. 81 users participated in the data collection sessions through a dedicated mobile app installed on the user's device. The user was asked to perform a series of tasks. In order to check whether the model can differentiate between the notion of user and device, two dedicated sessions were carried out by another person (imposter) on the same exact device at the time of data collection. Data was collected from 15 background sensors as well, i.e., touchscreen, accelerometer, and so on. For evaluation, **Long-Short Term Memory (LSTM)** architecture was used.

Al-Obaidi and Al-Jarrah [7] proposed a classifier leveraging a median-based method for keystroke authentication on mobile devices. The classifier was used as an anomaly detector on the SU dataset [14]. For the SU dataset, the CMU password ("tie5Roanl") was used. There were both timing and touch features of mobile devices in this dataset. There were 11 characters in the password including the Enter key resulting in 71 features in total (timing and touch screen features). 42 users took part in the data collection typing the same password 51 times. 34 entries were used for training and 17 for testing. Data collection was performed on Android devices, a tablet, and a mobile phone. The proposed statistical classifier model used the median of the training features. The testing features were authenticated against the training features. There were three timing features: **Hold (H)**, **Up-Down (UD)**, **Down-Down (DD)**, and two touch screen features: **Pressure (P)** and **Finger Area (FA)**. The first five entries of each subject were used to create an imposter set. Al-Obaidi and Al-Jarrah achieved an EER of 8.53% for 41 timing features and an EER of 6.79% for both timing and touch screen features.

Al-Obaidi and Al-Jarrah [6] proposed a keystroke authentication system for use on mobile devices. They adopted an anomaly detector leveraging statistical distance-to-median that used both the timing and touch screen features. The goal was to develop a suitable keystroke dataset for mobile devices and evaluate the performance of the distance-to-median anomaly detector. They performed their experiment on Nexus smartphones and tablets and used five features for the authentication system: H, Latency or UD, DD, P, and FA. The system consisted of two phases. In the training module, the required tasks were: (1) Registration of users through user-id and password, (2) Selection of the number of attempts of password entry and the pass-mark, (3) Password typing for the selected number of times, and (4) Template generation. In the testing phase, the test vector was compared against the template stored in the database. The test vector was generated from the login attempt. The data was collected from 56 users each typing a 10-character password 51 times. For a different pass mark per subject, they achieved an EER of 0.049. For a global pass-mark, Al-Obaidi and Al-Jarrah achieved an EER of 0.054 and implemented three verification models based on distance to obtain their results. They obtained an FRR (false rejection rate) of 5.6% for the FAR (false acceptance rate) of 5%.

Kim and Kang [59] proposed a novel keystroke dynamics-based authentication method for mobile devices named FACT (free text, accelerator, coordinate, and time). In order to evaluate their proposed model, Kim and Kang collected a novel keystroke dataset based on mobile devices. They used an Android-based application for the data collection. A total of 50 subjects participated in the data collection. 20 free-text scripts were typed by each subject and each script contained about 200 keystrokes. Special characters were not included in the scripts. The subjects provided the data in both English and Korean. 17 features were extracted from the raw data including not only the typing pattern but also hand size, finger length, fingertip size, and muscle flexibility.

Table 3. A Summary of Keystroke Authentication Algorithms, the Datasets that Were Used to Evaluate the Algorithms, and the Results Obtained Using These Algorithms

Algorithm	Dataset	Result
Distance Metric Fusion [70]	Murphy et al. [76]	EER-3.6%
Gaussian Mixture Model [21]	Vural et al. [112]	EER-0.08%
Sorted Time Mapping Technique along with Neural Networks [5]	Ahmed and Traore [5]	EER-2.46%
Instance-Based Tail Area Density Metric combined with Fused Matching Score [16]	Murphy et al. [76] and Sun, Ceker, and Upadhyaya [104]	EER-7.8% and 3%
Novel Statistical Model [35]	Foresi and Samavi [35]	FAR-0.0% and FRR-2.54% and 2.87%
Combination of MLP and Trust Algorithm [82]	Popovici et al. [82]	Classification rate-80%
Transfer Learning Techniques [24]	Vural et al. [112]	EER-19.47%
XGBoost [97]	Killourhy and Maxion [58]	Accuracy-93.60%
Event Sequences [106]	Syed, Banerjee, and Cukic [106]	Increased Effectiveness
ECM-ELM Classification Model [84]	Killourhy and Maxion [58]	Stable and Good Accuracy(86.97%)
Authentication without Template [96]	Singh et al. [96]	SFR and SFA - 87
Triboelectric Device [118]	Wu et al. [118]	EER-1.15%
Recurrent Neural Network [60]	Killourhy and Maxion [56]	EER-0.136
mRMR Feature Selection Technique [61]	Krishnamoorthy et al. [61]	Accuracy-0.9740 and F1 score-0.9701
Nearest Neighbor(New Distance Metric) and Outlier Removal [122]	Killourhy and Maxion [58]	EER-0.084 and ZMFAR-0.405
TypeNet [1]	Acien et al. [1]	EER-2.2%
TypeFormer [100]	Aalto mobile [80]	EER-3.25%
DoubleStrokeNet [78]	Aalto desktop [32] and mobile [80]	EER-0.75% and 2.35%

The proposed FACT method achieved an EER lower than 1%. Kim and Kang made their collected keystroke dataset public [59].

4 Keystroke Authentication Algorithms

The authentication algorithm is a crucial part of any keystroke-based authentication system. This section describes state-of-the-art keystroke authentication algorithms. A summary of keystroke authentication algorithms is shown in Table 3.

4.1 Statistical Algorithms

Statistical algorithms are easy to implement, model, and explain. They are deterministic and also do not require heavy computing power and/or memory. However, they require manual extraction of features which could be difficult. Below, we describe common statistical algorithms used for keystroke dynamics.

4.1.1 GMM. GMM is a statistical method based on the weighted sum of **probability density functions (PDFs)** of multiple Gaussian distributions [31, 112]. GMM generates a vector of mean values corresponding to each component and a matrix of covariance which includes components' variances and the covariances between each other. The parameter set λ is used for expressing GMM. The parameter λ comprises of component weights w_i , mean vector $\vec{\mu}_i$ and covariance matrix Σ_i as shown in equation $\lambda = \{w_i, \vec{\mu}_i, \Sigma_i\}, i = 1, \dots, M$. The parameters are estimated using the iterative **expectation-maximization (EM)** algorithm [30]. In every iteration, the parameter (λ) is updated if the iteration yields a higher likelihood and fits the distribution of the training dataset.

Given a set of reference samples λ (also known as profile), the mixture density for the keystroke input sample $\vec{x}$ is defined as the weighted linear combination of M pure Gaussian distributions as shown in the equation: $p(\vec{x}|\lambda) = \sum_{i=1}^M w_i * p_i(\vec{x})$; where $p_i(\vec{x}) = \frac{1}{(2\pi)^{D/2} |\Sigma_i|^{1/2}} \exp\{-\frac{1}{2}(\vec{x} - \vec{\mu})^T \Sigma_i^{-1} (\vec{x} - \vec{\mu})\}$.

4.1.2 Euclidean Distance. Euclidean distance is the shortest distance between two locations (points) in Euclidean space and is found in equation $D = \frac{1}{N} \sqrt{\sum_{i=1}^N (\mu_{g_i} - x_i)^2}$; where N represents the number of digraphs shared between the test sample and the profile, x_i is the individual test graph duration for the i^{th} shared graph in the test sample, and μ_{g_i} is the mean of the i^{th} graph in

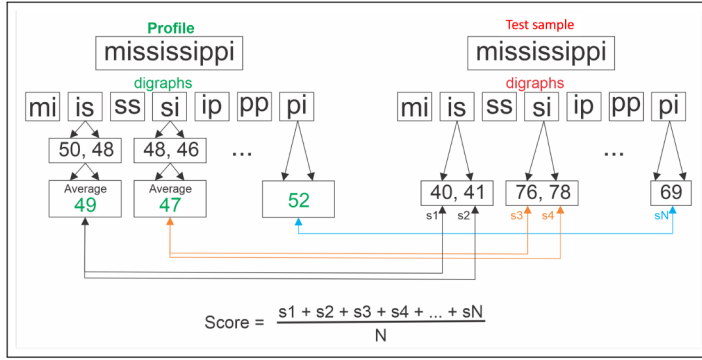


Fig. 4. Instance-based scoring procedure for sample text ‘mississippi’. The graphs in the test sample are compared with those of the profile sample and similarity scores are calculated. The average of the scores is returned as the final score.

the profile [58, 107]. The Euclidean distance is an instance-based similarity metric and comparison is done on a single occurrence of a graph from the test sample to the reference profile, as shown in Figure 4. The Euclidean distance assumes that the distribution of the graphs to be compared has the same variance. Hence, for two graphs with significant differences in variance, the Euclidean distance result is less accurate. Furthermore, if a correlation is present between the features, which is generally the case in real-world datasets, the Euclidean distance between a point and the mean of the points can give less accurate or misleading data about the distance between points. A reason for this is that Euclidean distance is a distance between two points only and fails to consider the relationship between a point and other points.

4.1.3 Manhattan and Scaled Manhattan Distance. The Manhattan distance, also known as the city-block distance, has been commonly used by keystroke researchers [18, 57, 109] and can be calculated in equation $D = \frac{1}{N} \sum_{i=1}^N \|\mu_{gi} - x_i\|$. N is the number of digraphs shared between the test sample and the profile, x_i is the individual test graph duration for the i^{th} shared graph in the test sample, and μ_{gi} is the mean of the i^{th} graph in the profile. The Manhattan distance also follows an instance-based procedure (Figure 4). Similar to the Euclidean distance, the Manhattan distance assumes that the test and profile graphs have the same variance, which is often not the case. Hence, the Scaled Manhattan distance, a modified version, is used. It can be calculated using equation $D = \frac{1}{N} \sum_{i=1}^N \frac{\|\mu_{gi} - x_i\|}{\sigma_{gi}}$. The Scaled Manhattan distance is similar to the Manhattan distance, except that the former is divided by the standard deviation (σ_{gi}) of the i^{th} graph in the profile [58].

4.1.4 ITAD. Ayotte et al. [16] proposed the ITAD metric, a novel instance-based graph comparison algorithm, to decrease the required number of keystrokes for authentication. The ITAD metric is also an instance-based similarity metric (Figure 4). This metric uses the tail area under the PDF or the percentile value of each test graph in the profile sample. Alternative to the instance-based algorithm is the distribution-based algorithm [16]. Distribution-based algorithms compare the PDF of each graph in the test and profile samples. Unlike distribution-based which requires that there are at least 4 instances of a graph in the test and profile samples before they can be compared, instance-based algorithms (such as ITAD) can compare graphs in the test and profile samples even when there is just a single instance of that graph in the test sample. ITAD can be calculated as shown in Equation (5), where CDF_{gi} is the empirical cumulative distribution function of the i^{th} graph in the gallery (profile), M_{gi} is the median of the i^{th} graph in the gallery, x_i

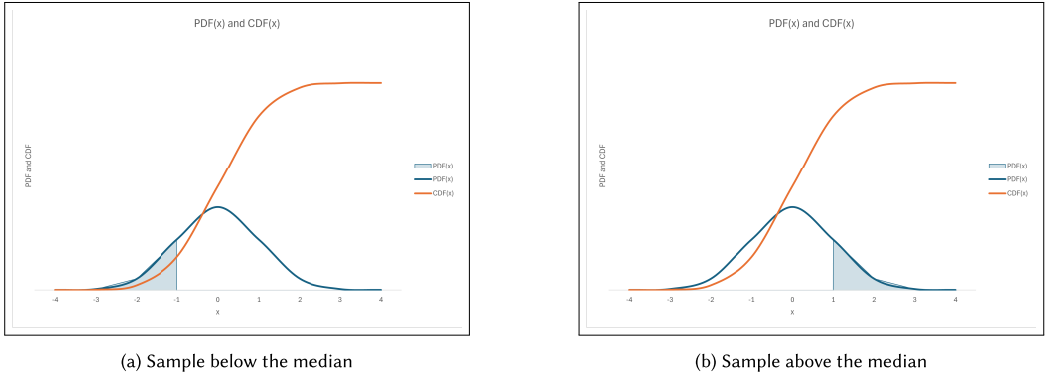


Fig. 5. Graphical representation of how the ITAD metric is computed from the PDF and CDF. In (a), the sample is below the median and the ITAD metric is determined by the left tail area of the PDF. If the left tail area is x , then the x point of the CDF (on the left) indicates the ITAD metric score. In (b), the sample is above the median and the ITAD metric is determined by the right tail area of the PDF. If the right tail area is y , then the $(1-y)$ point of the CDF (on the right) indicates the ITAD metric score.

is the individual test graph duration for the i th graph in the test sample that was shared with the gallery, and N is the number of graphs shared between the test sample and the gallery. The ITAD metric scores for N graphs are averaged and a single similarity score is returned as given in Equation (6).

The illustration in Figure 5 depicts the computation of the ITAD metric using a graphical representation. The ITAD metric is calculated as the area of the tail of the PDF. If the sample falls below the median, the ITAD metric is determined by the tail area on the left, while if it is above the median, it's determined by the tail area on the right. ITAD performs better than any distance metrics (such as the Scaled Manhattan) which are based on mean and standard deviation of graphs and are affected by outliers. ITAD metric scores range between 0 and 0.5 and can be interpreted as a measure of similarity. The larger the score is, the closer the test sample is to the profile.

$$S_i = \begin{cases} CDF_{gi}(x_i), & \text{if } x_i \leq M_{gi} \\ 1 - CDF_{gi}(x_i), & \text{if } x_i > M_{gi} \end{cases}, \quad (5)$$

$$\text{Similarity Score} = \frac{1}{N} \sum_{i=1}^N S_i. \quad (6)$$

4.1.5 Mahalanobis Distance. Mahalanobis distance is an example of a distribution-based algorithm. It is an effective multivariate distance metric that measures the distance between a point and a distribution [58, 86, 121, 122]. It takes into consideration the correlation between features which is more suitable when features are not independent. It is best for finding multivariate outliers and finds its applications in multivariate anomaly detection, classification, and one-class classification. The Mahalanobis equation is: $D = (X - \mu_t) \cdot C^{-1} \cdot (X - \mu_t)$; where X is the feature vector of a test sample, μ_t is the mean feature calculated from the profile template, and C^{-1} is the inverse covariance matrix of the user profile template.

4.1.6 Gunetti and Picardi's Metric. Gunetti and Picardi's free-text algorithm [46] combines typing speed (A-measure) and the degree of disorder (R-measure) to measure similarity. The 'A' measure represents the distance between typing samples S_1 and S_2 in terms of n -graphs, where n is the number of consecutive keystrokes ($n = 1$ is monograph, $n = 2$ is digraph, $n = 3$ is trigraph, etc.), is

given in equation -

$$A_{t,n}(S_1, S_2) = 1 - \frac{\text{Number of similar n-graphs between } S_1 \text{ and } S_2}{\text{Total number of n-graphs shared by } S_1 \text{ and } S_2}, \quad (7)$$

where t is a constant for determining n-graph similarity. For example, let G_{S_1, L_1} and G_{S_2, L_2} be the same n-graph occurring in typing samples S_1 and S_2 , with duration L_1 and L_2 , respectively. We say that G_{S_1, L_1} and G_{S_2, L_2} are similar if and only if $1 \leq \max(L_1, L_2)/\min(L_1, L_2) \leq t$.

The 'R' measure on the other hand, quantifies the degree of disorder between two sequences. Given an array V of K elements, a simple measure of the degree of disorder of V with respect to V' (its ordered counterpart) is computed as the sum of the distances between the position of each element in V and the position of the same element in V' .

4.2 Machine Learning-based Algorithms

Machine learning methods have been increasingly employed for keystroke dynamics. Here, we will detail the most commonly used machine learning-based methods that are used for keystroke dynamics.

4.2.1 Supervised Learning. Supervised Machine Learning results in a model that predicts the outcome/label of a given data after the model has been trained with labeled data. The model at training has access to the label of the input (training) data which is used for computing training errors through the loss function. Based on training errors, the model backpropagates and updates the weights. When model training is complete, it is used to predict unseen input data in the absence of labels. Model accuracy is represented by the number of accurate predictions over the total number of predictions.

Supervised Machine Learning can be further divided into classification and regression. Classification uses an algorithm to accurately predict a class label on test data e.g., classifying user behavior as genuine or imposter. Binary classification refers to a problem involving two classes and multi-class classification refers to a problem with greater than three classes. On the other hand, regression makes *numerical* predictions such as predicting the weather or average salary. In keystroke dynamics, the focus is either to verify if the user is who he/she claimed to be (authentication) or to identify the user to whom the query sample belongs to (identification). Both scenarios belong to the classification type, hence, keystroke dynamics uses the classification techniques.

SVM. An SVM is a type of supervised learning model leveraged for classification problems. The SVM algorithm computes hyperplane(s) that uniquely separate classes of points of data. The hyperplane, also known as the decision boundary, is an N -dimensional space plane, where N is the number of features in the dataset. Of several hyperplanes possible, the goal is to compute a boundary with the maximum margin that separates the classes (legitimate users from impostors) [41, 61, 97, 118]. This algorithm derives its name from support vectors, which are data points that are closer to the hyperplane and decide the orientation of the plane.

In keystroke dynamics, it is assumed that the keystroke samples in the instance of enrollment and verification have an equal distribution, which is not usually the case, and as a result, lower prediction accuracy is obtained from most traditional machine learning classifiers. In this regard, SVM has support for adaptive learning [24], where an existing SVM classifier (source) is adapted to another SVM classifier (target) based on some projection techniques. Knowing that there exist variations in users' typing patterns at every sample, especially when the samples have a wide time margin (weeks, months, or years apart), the adaptive SVM modifies the hyperplane of a pre-trained classifier using the new data points with minimum retraining and therefore performs better than

the regular SVM trained on the same amount of labeled data. The SVM classifier is a commonly used classifier in keystroke dynamics, especially on a small feature set dataset [22, 39].

Naive Bayes (NB). The NB classifier is a probabilistic machine learning model used for classification and based on the Bayes theorem with “naive” independent assumptions between features [15, 28, 49]. The Bayes’ theorem finds the probability of an event a occurring given that the probability of another event B has already occurred, as shown in equation $P(a|B) = \frac{P(B|a) \times P(a)}{P(B)}$; where $P(a|B)$ is the posterior probability, $P(a)$ is the prior probability of the label class, $P(B)$ is the prior probability of the predictor and $P(B|a)$ is the likelihood. The NB classifier offers several advantages, such as being less computationally demanding compared to other traditional machine learning models, and performing well on small training data when the assumption of feature independence holds. Additionally, it can be used for solving multi-class prediction problems, which are identification problems that involve predicting one of several possible classes (or outcomes) for a given input. In keystroke dynamics, where dependencies between features may exist, the NB classifier may not be as effective and is therefore less commonly used.

Tree-Based Models. Tree-based models are a type of supervised machine learning algorithm used for classification. They use a set of conditional statements to recursively split training data into subsets. The end result model is a roadmap of logical decisions that describes the dataset. Tree-based models are easy to implement and interpret, less computationally intensive, and are popularly used in keystroke dynamics because of their non-parametric attributes, making it preferable even for non-Gaussian distributed data [94]. Tree-based models are tolerant to outliers and intra-class variation problems, work with both numerical and categorical variables, and require fewer data preprocessing steps such as variable transformations (scaling, normalization, etc). However, they are prone to overfitting and high variance. Popular tree-based methods used in keystroke dynamics are **random forest (RF)** [10, 16], gradient boost and **extreme gradient boost (XGBoost)** [25, 97]

K-Nearest Neighbors (KNN). KNN is a supervised machine learning classifier (although it can also be unsupervised) that classifies new data points based on their proximity with data points of already known classes. KNN works on the assumption that data points of similar classes are closer to each other and can be found near each other [15, 51, 99]. During testing the distances between a new point of data and prior points (neighbors) are calculated. A class label is assigned to a new data point based on a majority vote between its KNNs, where K is the number of neighbors that will be checked to determine the class to assign for the new query point. Generally, the optimal value of K is found experimentally, as a low value of K results in low bias but high variance, while a high value of K gives high bias and low variance. Distance metrics (such as the Euclidean distance, Manhattan distance, Hamming distance, etc.) are used to calculate the distance between the query point and other data points. These distance metrics form the decision boundaries. KNN is known as a lazy learner as it does the nearest neighbor calculation at every run time. Because of this, it is used only on low-dimensional data and has a high computational cost when used on high-dimensional data.

Multi-Layer Perceptron (MLP). MLP is a type of supervised learning algorithm consisting of input, output, and several hidden layers (those in between the input and output layer). The input data to be processed is passed to the input layer, the output layer does the classification, and the hidden layers (non-linear layers) perform the computational work. Therefore, data flows in the forward direction from the input layer to the output layer. MLP is a neural network with multiple layers and each layer is a combination of neurons such that the output of some neurons is passed as input to other neurons in the next layer. The number of neurons and hidden layers used is

dependent on the type of classification problem being solved. MLP is capable of learning non-linear models, but it requires several hyperparameter tuning such as the number of hidden neurons, hidden layers, and iterations. Hence, it is computationally intensive and requires both genuine and imposter data to train. The neural network is capable of producing much better performance than statistical algorithms and any other machine learning algorithms when working with big data that has both genuine and imposter samples [5, 47, 82, 103].

4.2.2 Unsupervised Learning. Unsupervised learning is a type of ML used for identifying patterns in datasets that contain unlabeled data points, whereas with supervised learning, the data is labeled and the algorithm is trained on this data to make predictions. In keystroke dynamics, unsupervised learning algorithms are often used for outlier detection. Some of the unsupervised learning algorithms used in keystroke dynamics include One-class SVM (OCC), K-NN, and K-Means [55, 115]. These algorithms are useful in situations where obtaining labeled data is difficult, costly, or impossible, and the ground truth (label) is unknown.

4.3 Deep Learning-based Algorithms

Deep learning algorithms have become the new state-of-the-art in the field of keystroke dynamics. Deep learning is a machine learning technique that learns features automatically from data. The data in this case are sequential keystroke data. Deep learning, when modeled properly, is robust to natural variations in sequential or temporal data. In keystroke dynamics research, usually a conventional machine learning algorithm works as a binary classifier whereas a deep learning algorithm works as a global model that generates embeddings for all subjects.

Deep learning is data-hungry and its performance is partially directly proportional to the volume of the training data used, and as such, its application in keystroke dynamics has been limited due to the lack of large publicly available datasets. However, with the availability of large keystroke datasets such as the Aalto [32, 80] and Clarkson II [53] dataset, this option is now being explored [1, 2]. Popular deep neural network types are the **convolutional neural network (CNN)** and **recurrent neural network (RNN)**.

4.3.1 CNN. CNNs [45] are deep learning algorithms that take in an input image and learn to identify key features in the image much like human vision. The basic CNN architecture is comprised of several layers such as the input layer, the convolution layer where high-level features are extracted from the input image, the pooling layer where the spatial size of the convolved feature is reduced, and the fully connected or dense layer where the result is flattened out and passed through an activation function that outputs the final classification result [36]. Basically, CNN uses convolution, also known as a sliding window (“filter”) that passes over the image, extracting key features and feeding them to lower layers in the network. A CNN, when trained with enough data, can successfully capture the spatial and temporal information in the data. Although CNNs were originally created for image classification where pixels form a two-dimensional grid, it has found its usefulness in keystroke dynamics [23, 65, 119] and several other fields.

To leverage the CNN for keystroke analysis, keystroke data are grouped into sequences, with each sequence containing a fixed length (L) of sequential keystroke data. Each sequence is then converted into a two-dimensional grid of specified dimension $L \times B$ (similar to an image), and all sequences are passed as input into the CNN model to be trained, where L is the sequence length (also known as timesteps) and B is the number of features. The overall input size is represented by: samples (N), timesteps (L), and features (B) where samples (N) is the total number of sequences created from the dataset. There are two main types of CNN that can be used: the **two-dimensional CNN (2D-CNN)** and the **one-dimensional CNN (1D-CNN)**. The 2D-CNN is the standard CNN where the kernel (filter) slides along 2 dimensions on the input data and extracts the spatial features

[65]. The 1D-CNN on the other hand, has a kernel that covers all the features and slides along one dimension [67]. Both the 1D-CNN and 2D-CNN have shown to be effective on time-series sequential data such as the keystroke data, however, the 1D-CNN is preferable and has produced better results in the literature. This is because 1D-CNNs are designed to handle sequences of one-dimensional vectors, while 2D-CNNs are designed to handle images or data represented in two dimensions. In keystroke dynamics, the data is usually represented as sequences of time-based features including but not limited to HT and inter-key time. 1D-CNNs can capture the temporal patterns in the sequences of these features, whereas 2D-CNNs would not be well-suited for this type of data. Additionally, due to the sequential characteristic of keystroke data, extensive padding would be required to produce a 2D representation, which would be computationally expensive and may negatively affect model performance. 1D-CNNs are computationally more efficient and better suited for keystroke dynamics data, as they can effectively handle the sequential patterns in the data without the need for padding. Although CNN is less computationally intensive than a RNN, it can not process all keystroke data in a sequence simultaneously and it is unable to capture the sequence order information like RNN would.

4.3.2 RNN. The RNN [69] is a deep learning algorithm best used for sequential or time series data such as keystroke data. An RNN keeps track of information from previous inputs through its memory gate and decides which information in the past should be remembered or forgotten based on the current input and output. As a result, unlike other deep neural networks, the output of RNN depends on previous information in the sequence, especially for a unidirectional RNN. However, a bidirectional RNN can produce an output that depends on both the past and future information in the sequence.

LSTM is a popular RNN architecture, introduced by Hochreiter and Schmidhuber [50]. It is used to model long-distance relations in input data. The main benefit of LSTM is the inclusion of a cell state, which simply acts like a memory chain by storing information from past states. There are three gates in LSTM: an input gate, an output gate, and a forget gate. The three gates act as logic control within the model used to predict the model's output. The forget gate is computed using equation $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$; where σ is the sigmoid function, h_{t-1} is the output from previous step, x_t is the input from current step, and W_f and b_f are the weight matrix and bias for the forget gate. At the input gate, the computations are carried out with these equations $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$; $\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$. Results are then used to update the cell state using equation $C_t = f_t \cdot C_{t-1} + i_t \cdot \hat{C}_t$; where C_{t-1} represents the state of the previous cell. The final output for the current step is calculated using these equations $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$; $h_t = o_t \cdot \tanh(C_t)$.

This variant of RNN has been widely used in keystroke dynamics for both authentication and identification [1, 29, 60]. Acien et al. [1] created a deep learning architecture that is quite different from the conventional LSTM architecture with the **Siamese Neural Network (SNN)** and they called it TypeNet.

SNN is a class of neural network architectures that contain two or more mirrored sub-networks. A major difference between SNN and other traditional LSTM models is that an SNN is designed to find the degree of similarity of the inputs to the sub-networks. Hence, it is able to predict new classes of data from users not seen during training without the need to retrain the model. The architecture achieved state-of-the-art performance in keystroke dynamics authentication with EER of 2.2% and 9.2% on Aalto desktop [32] and mobile [80] datasets, respectively.

Gated Recurrent Unit is also a variant of RNN with similar properties to an LSTM [27]. Instead of a "cell state" and three gates used in LSTM, GRU uses hidden states and has two gates: a reset gate and an update gate. The reset and update gates control which information to retain. The GRU

is without the memory unit which makes it expose the full hidden content without any control. With fewer gates, GRU trains faster, is less complex, and performs better than LSTM, especially on smaller datasets [60, 64]. In relation to keystroke dynamics, Koboжек and Saeed [60] contrasted the performance of GRU and LSTM on the same dataset using two scenarios, the first was using only the dwell time, and the second was using all the data. They reported that the GRU (15% EER) outperformed LSTM (21.9% EER) when only the dwell time is used, but the LSTM (13.6% EER) outperformed GRU (22.4% EER) when all the data was used.

4.3.3 Transformer. Stragapede et al. [100] developed a novel Transformer architecture named TypeFormer for mobile keystroke dynamics for user authentication purpose. Their Transformer network was based on a two-branch architecture: Temporal and Channel Modules enclosing two LSTM recurrent layers, **Gaussian Range Encoding (GRE)**, a multi-head Self-Attention mechanism, and a Block-Recurrent Transformer layer. TypeFormer could map slices of keystroke sequences into a feature embedding space. Same subject representations of sequences were closer than those of different subjects. The triplet loss function was used to train TypeFormer and the similarity of the feature embeddings was measured with Euclidean distance. The pre-processed Transformer input sequences were fed to the branches. A GRE was used for modeling the input sequences in both branches; the purpose was to preserve the information position. The Temporal Module consisted of three ordered sets of layers. Each of the sets of layers was composed respectively by N, R, and M layers. The N and M layers were made of two sub-layers: a multi-head Self-Attention mechanism, and a multi-scale keystroke LSTM RNN layer. R recurrent layers were included between the N and M layers. The Channel Module consisted of H layers similar to the N and M layers of the Temporal Module. There was a convolutional layer at the end of each of the Modules. The output feature embedding vector P was generated by concatenating the similarity of the output features. The output feature embedding vectors were compared through the Euclidean distance for the authentication task. Stragapede used TypeFormer on the Aalto mobile keystroke database [80] and achieved an EER of 3.25%.

Neacsu et al. [78] proposed a novel Transformer-based keystroke authentication method named DoubleStrokeNet. In order to generate user embeddings, this model used the temporal features of bigrams. A Transformer encoder was the main part of the DoubleStrokeNet architecture. The Transformer encoder functioned as a discriminator between legal and imposter users. It could also differentiate between original and replaced key bigrams. The transformer encoder was able to extract relevant contextual information within data sequences and encode key bigrams based on context-aware embeddings. Neacsu et al. used Aalto desktop [32] and mobile [80] keystroke datasets for their experiments and achieved EER values of 0.75% and 2.35% for physical and touch-screen keyboards, respectively.

5 Keystroke Feature Engineering Approaches

Throughout the following section, the impact of different keystroke feature engineering techniques is described. The performances of keystroke authentication algorithms depend largely on keystroke data/keystroke features that are used for authentication, so it is important to determine which features give the best results.

5.1 Effects of Text Filtering

Huang et al. [54] investigated the effect of gibberish text on the performance of keystroke dynamics. Noise or gibberish text can originate when a user plays a computer game or does some unnecessary task. Huang et al. decided that the unnecessary text should be filtered out due to its negative impact on authentication performance. For the evaluation of their hypothesis, they used a novel dataset

that was collected when the users were performing normal day-to-day activities. There was no restriction or control over the users. They were free to type anything in order to analyze the noise keystrokes. Huang et al. used a keystroke logger to collect the data of 60 users in a campus student computer laboratory. Each user provided at least 11,000 keystrokes. They used Gunnetti and Picardi's algorithm [46] for evaluation. They used samples of 1,000 keystrokes to create a user profile. Each user profile consisted of 10 samples with the rest reserved for testing. They also used the "zone-of-acceptance" algorithm by Leggett et al. [62]. From the dataset, they detected four types of gibberish keystrokes: repeating characters, gaming patterns, long strings with few distinct characters, and long strings with no white spaces or separators. They then used two types of algorithms in order to remove gibberish keystrokes. The first one was based on regular expressions that identified four patterns of gibberish keystrokes. The second one was a spell checker that checked for correct English words. They also used context-based filtering and determined that 23.3% data in their dataset was gibberish. The evaluation showed that though gibberish keystrokes did not have much effect on FAR, they negatively impacted FRR. Spell checker filtering showed better performance than regular expressions, but the best result was achieved by combining both filters.

5.2 Effect of Data Size

Huang et al. [52] analyzed the performance degradation of keystroke authentication systems by experimenting with data size. They analyzed the impact of reference profile size and test sample. They used two algorithms for their evaluation process. The first one was Gunetti and Picardi's algorithm [46]. The second one was the "zone-of-acceptance" algorithm by Legget et al. [63]. They used the dataset developed by Vural et al. [112] for their evaluation and performed two experiments. In the first experiment, they detected all instances of digraphs and trigraphs in the previously collected user's data, then created a reference profile and a test sample of a specific size. Every instance of data contained within the reference profile was divided into five samples in order to implement the Gunetti-Picardi algorithm. The dataset contained the data of 39 users. They performed the tests 50 times to avoid errors. In the second experiment, each user's data was divided into groups of 1,000 digraphs. The range of reference profile size was from 6,000 digraph instances to 17,000 digraph instances. Only 25 users had enough data to perform the tests. From the first experiment, the results indicated that the FAR value improved when the reference profile size was constant and the test sample size increased. The results also showed that keeping the test sample size constant and increasing the reference profile size did not necessarily improve the FAR. The **Imposter Pass Rate (IPR)** improved when the reference profile size increased. The IPR values did not decrease for smaller test samples. In the second experiment, Huang et al. used the "zone-of-acceptance" algorithm. The results showed that both FAR and IPR improved with an increasing the reference profile size. They also found out that the reference profile should consist of a minimum of 10,000 keystrokes or more to achieve good performances.

5.3 Impact of Faulty Users' Data

Ozbek's goal was to improve the classification performance by removing the keystroke data of the users who degrade the performance [79]. Ozbek used two benchmarking databases with different numbers of users and passphrases for the evaluation.

- (1) GREYC Keystroke Data: 133 participants typed the passphrase "greyc laboratory" on an AZ-ERTY keyboard for the collection. There were 7,555 attempts [40]. For this dataset, Ozbek removed some of the users with a low number of captures leaving only the 100 participants with more than 50 attempts.

- (2) CMU Keystroke Data: 51 participants typed the password “tie5Roanl” with 400 attempts per user [58].

The data of the two datasets were used as they were and no features were removed. Also, no dimension-reduction techniques were implemented to improve the performance. To evaluate the performance Ozbek used SVM, decision tree, and KNN classifiers. For these three classifiers, each keystroke datum was divided into training, testing, and validation sets. 70% of the data was used for training, 15% was used for testing, and 15% was used for validation. No cross-validation technique was applied. Ozbek used a histogram to detect the faulty users. Re-classification of the data was performed with the remaining users. For both datasets, Ozbek achieved higher accuracy by eliminating the users with misclassifications at the time of training. First Ozbek removed the worst five users which improved the performance. Ozbek achieved better results by removing more users having misclassifications. However, removing the users from the database was not a standard option. The main goal was to demonstrate that the worst users could be recognized and eliminated if required.

5.4 Impact of Non-Conventional Keystroke Features

Alsultan, Warwick, and Wei [12] used non-conventional keystroke features for the authentication of users. In the case of the non-convention features, the focus was mainly on the overall typing patterns. The percentage of performing certain actions was considered, such as editing actions, general typing actions, and so on. The goal was to understand the typing behavior of a user. Two main features were considered—semi-timing and editing features. Semi-timing features were different from the common timing features used in previous experiments. These features were calculated for greater amounts of time. The semi-timing features that they used were: (1) **Word-per-Minute (WPM)** = Number of words/Total typing time in minutes, (2) **negUD** (negative Up-Down) = Number of negative UDs/Total number of keypairs, (3) **negUU** (negative UP-Up) = Number of negative UUs/Total number of keypairs. Editing features were characteristics such as typing error frequency, text editing, and so on. The editing features were: (1) Error rate, (2) CapsLock usage, and (3) Shift key usage. Thirty participants provided data for this study and participants had to complete eight typing tasks. The tasks included copying the text of 1,000 characters. The text was taken from the Guardian newspaper. In the text, there were numbers, both upper and lower case letters, and punctuation marks. They used a C++ based GUI program for the data collection. The participants could download the application on their own machines. As the user’s profile, a feature vector of nine features was generated and saved in the database. Out of eight typing tasks, each was accomplished with a typing sample from a single source. Features were identified and extracted independently of each typing task.

In the analysis phase, eight samples per subject were used for training and testing of the classifier. As a classifier, decision trees were used in this research. Cross-validation was used in the classification process. In their experiment, Alsultan, Warwick, and Wei used eight samples to perform eight cross-validation experiments. They used seven samples for training and one sample for testing. They used the statistics toolbox in Matlab and two error rates to represent the results - FAR and FRR. They obtained low error rates in this study and computed the rates of error for a variable number of participants. For the decision tree classifier, they obtained a FAR of 0.007, 0.0104, and 0.0109 for 15, 25, and 30 participants, respectively. They obtained a FRR of 0.1, 0.25, and 0.28 for 15, 25, and 30 participants, respectively.

The focus was on true intruder detection in the second part of this study. In this part, typing samples from unknown users were used to test the system. Here, they used binary classification. For the binary classification, they used the training data from 25 legal users and the testing data

from five intruders. Each intruder generated three typing samples to test the system. For the decision tree classifier, they achieved a FAR of 0.011 and a FRR of 0.375 for 25 genuine users without any intruders. For the SVMs, they achieved a FAR of 0.0112 and a FRR of 0.49. When the data of the five intruders was tested against the genuine users' data, they got the same FAR in the case of SVMs. In this study, the FRR improved to a value of 0.28 utilizing non-conventional characteristics. The lowest FAR was 0.011, which was also comparable to the results generated by conventional features.

6 Keystroke Authentication on Touch Screen and Mobile Devices

As people are increasingly dependent on touch screen devices such as tablets and mobile phones, it is necessary to implement security and authentication systems based on these devices. This section describes various aspects of keystroke authentication on touch screen and mobile devices. The rest of this section is provided as supplementary material.

7 Other Applications of Keystroke Dynamics

The number of applications that rely on input from a computer keyboard is increasing as computers have become more commonplace in the home, workplace, and academia. People have become more comfortable with typing on keyboards. In fact, today's self-taught typists type almost as fast as touch typists [66]. This increase in the number of people who are typing on a keyboard creates additional opportunities to apply keystroke dynamics to domains not previously envisioned. An authentication API for adopting keystroke authentication is [typingdna](#). This keystroke authentication API can be used to secure logins, enforce password resets, and authenticate subjects.

The examples mentioned in this section are examples of domains where fixed-text and free-text keystroke dynamic analysis can be applied.

7.1 Educational Level Classification

Tsimperidis et al. [111] used keystroke dynamics for the detection of one's level of education. They proposed the randomized **radial basis function network (RBFN)**, which is a novel machine learning model. Their model is capable of identifying the educational level of the keyboard user. Their model is the first model leveraging keystroke biometrics for the detection of one's education level. Their experiment was performed in two steps. First, free-text unrestricted data were collected from the participants for ten months. The second step was developing the R^2BN model. The computational time of the R^2BN model was very high. In order to reduce that time, they decreased the size of the dataset without degrading the performance of the model. A filter-based random subfield data condensation approach was proposed. 242 log files were collected from five educational-level classes. There were 2,800 to 4,500 keystrokes in each log file. 162 features were extracted from the data. The data was fed into the R^2BN model. This model was developed by the randomized modification of a popular **radial basis function neural network (RBFN)**, proposed by Broomhead and Lowe [20]. The advantage of R^2BN model was its speedy convergence and smaller extrapolation errors due to a shallow-learning architecture. This model achieved more than 85% accuracy in the educational level classification task using keystroke dynamics.

7.2 Emotion Detection

Detecting emotions from your counterpart during electronic communications, such as e-mail or chat, is often difficult. Written words can be misinterpreted into thinking the other side is angry, or conversely, saddened, when in reality, the real emotion behind the discussion is unknown. Determining the emotion associated with the text can lead to improved communication. Feelings such as anger, sadness, disgust, fatigue, joy, fear, and indifference can be assumed from the way

a user types [77]. Associating emotions based on the way an individual types can reduce guesswork on the emotional state of the author and reduce confusion. Such a system can also serve as a warning for users to rethink or delay impulsively sending e-mails or other messages when angry, preventing regret.

Qi, Jia, and Gao [83] proposed an emotion recognition system based on piezoelectric keystroke dynamics. Piezoelectric materials are able to reflect the force detection sensitivity. Their experiment was simply a password entry. They used time and pressure dimension features. For emotion recognition, they utilized the discrete PAD 3-dimensional model. **International Affective Digitized Sounds (IADS)** (a standardized set of 110 digitalized sounds) was implemented to reveal emotions. In order to measure the amount of emotion induction, they implemented a PAD emotion scale. Their target was to classify four emotions—happiness, fear, sadness, and disgust. They obtained an accuracy of 78.31% using the RF Classifier. Their proposed approach proved to be a pragmatic one for emotion classification in practical applications.

Nahin et al. [77] identified the emotions of users by analyzing the textual patterns of the user's keystroke dynamics. They considered seven basic emotional classes, viz. anger, guilt, disgust, joy, fear, shame, and sadness. They used the categories from the **International Survey on Emotion Antecedents and Reactions (ISEAR)** [91]. There were two steps in their data collection process. (1) A Java-based software to collect fixed-text data, (2) An app written in C# to collect free-text data. After data collection, a software program written in C was used for keystroke feature extraction from the log files in order to separate feature files. 19 keystroke features were extracted from the collected data. Nahin et al. used the best seven features for analysis. Weka 3.6.6 was used for training. They developed different models for fixed-text and free-text data. For text pattern analysis, the ISEAR dataset was used [92]. In order to determine emotion class from raw text, they used VSM or term vector model [89]. For fixed-text analysis, the Weka tool was used [117]. These models were used to evaluate test datasets. They used these simple logistics, SMO, MLP, Random Tree, J48, and BF Tree for evaluation. They used these same methods for free-text analysis. They calculated the accuracies of emotion class recognition for both fixed-text and free-text. In the case of VSM, the raw data was used as input. VSM identified the emotional state of a sentence by analyzing the text. In the case of the final result, only those results were considered right that were the same for both keystroke dynamics and text pattern analysis. The combined results showed better accuracy than the separate results.

7.3 Age, Gender, and Demographics Prediction

7.3.1 Predicting Age and Gender. Pentel [81] used keystroke dynamics and mouse patterns to predict the characteristics of the user, including gender and age. Pentel collected data from six sources for this study between 2011–2017. To collect mouse movement events, a website that recorded mouse events was leveraged. To collect keystroke data, a key logging system was used, which was written in JavaScript. Pentel used five machine-learning algorithms implemented in Java, viz. Logistic Regression, Nearest Neighbor, SVM, RF, and C4.5. In the experiment, Pentel performed the classification of 10–15 years old separate from the other users in order to (1) balance groups with less under-sampling, (2) compare n-graph features based on time with features based on frequency, and (3) juridical purpose. For evaluation 10 fold cross-validation was used on all models. Pentel performed the training and validation 10 times. A single f-score value was used to present the results, which were calculated by taking the weighted average of both classes' f-score values. Two sample T-tests were performed between each age group to determine the variation between groups in typing speeds. Pentel computed the results of gender and age-based classification. For each classification, the baseline was 0.5. All the results were over baseline. For keyboard features, the best f-score for gender-based classification was 0.73, and the

best f-score for age-based classification was also 0.73. Pentel achieved the best results using RF for keyboard features. However, models trained on mouse features showed better results due to more mouse instances. Significant differences in typing speed were found between different age groups. T-test results showed that typing of the age group 16–29 was faster than other groups. Pentel compared the best time-based features with the frequency-based features of previous studies, and there was some overlap between the two sets.

7.3.2 Gender Recognition. Tsimperidis, Arampatzis, and Karakos [110] used keystroke dynamics to determine the gender of a user. They created a new dataset to identify the most useful features for gender recognition. There were three phases in their experiments. During the first phase, participants provided free-text data. In the second phase, features were selected via a selection algorithm. The features were sorted based on the information they contained. In the third phase, the results obtained by using different algorithms were compared. A free-text keylogger, called IRecU, was created for data collection. This keylogger was suitable for any Microsoft Windows-based devices. The participants had to install IRecU on their devices and use the keylogger during their daily activities without any restriction. 75 volunteers provided the log files containing the data. Data was collected over a period of 10 months. The dataset consisted of 248 log files; 125 labeled as “male” and 123 labeled as “female”. There was data from 2,800 to 4,500 keystrokes in the log files.

A software application named ISqueezeU was developed for feature extraction from the log files. The software application read the text files of the keylogger and then computed the average number of keystroke timings, including keystroke durations and down-down digraph latencies. They only considered the keys that appeared at least five times and the digraphs that appeared at least three times. Over 10,000 features were extracted. Due to the number of features, an automated selection method was required. The models that they used were: SVM, RF, NB, RBFN, and MLP. In their experiments, they tried to determine the gender of a user only by typing speed. The mean of all digraph latencies in each log file was computed, as it represented the typing speed of a user. Then they calculated and compared the male and female average values. For males, the average value of all digraph latencies was 373.04 ms, and the standard deviation was 135.26 ms. For females, the average value was 375.71 ms, and the standard deviation was 116.86 ms. As the values were very close for males and females, recognizing the gender of a user using typing speed was not possible. Keystroke dynamics dataset and several different sets of features were used in order to evaluate the performance of the machine-learning models, namely, RF, SVM, NB classifier, MLP, and RBFN. The calculated metrics were: the model accuracy (Acc.), the time complexity (TBM), the F1-score (F1), and the ROC index (AUC). The goal was to find out the optimal set of features. They performed numerous experiments on the five models mentioned earlier. Each time, they used a variable number of keystroke features. In every case, they used the polynomial kernel as the kernel type because of its better performance. Neural networks and SVMs performed better for gender recognition using features of keystroke dynamics. However, RBFN performed even better. They drew three conclusions based on their experimental results.

- (1) For all models, the highest accuracy was achieved before using the highest number of keystroke dynamics features.
- (2) In the range of 150–350 features, the five tested models achieved almost constant accuracy.
- (3) The RBFN model achieved 95.6% prediction accuracy for 350 features, which was the highest gender prediction rate using keystroke dynamics.

7.3.3 Predicting Typist Cognition and Demographics. Brizan et al. [19] used features based on keystroke dynamics, stylometry, and “language production” in order to recognize the cognition

and demographics of a typist. For their task, they used a dataset consisting of 350 users. The dataset contained free-text data. 1013 participants were included in the data collection phase for a total of two phases. 838 users participated in the first session, 491 users participated in the second session, and 486 users attended both sessions. Each user was provided a unique ID to identify them.

Demographics of age range, native language, gender, typing style, and handedness were collected. The features that they used were split into three categories: stylometry, language production, and keystroke dynamics, leading to a total of 2,381 features. The keystroke dynamics features that they analyzed were user typing speed, durations and frequency of pauses, pauses before specific keys, the average latency between any two keystrokes, key hold, and so on. The analyzed stylometric features were linguistic unit lengths, character type, consonant frequency, lexical diversity, lexical density, and so on. The language production features were part-of-speech pauses, punctuation pauses, misspelling pauses, revision features, typing burst features, and so on. They performed two sets of experiments.

- Cognitive Task Prediction: Four experiments were performed to identify the cognitive task. Four classifiers were used to predict the task type: SVM with a Linear Kernel, SVM with an RBF Kernel, NB, and AdaBoost with single split decision trees. The results showed that the SVM classifier performed better than other classifiers in predicting the cognitive task. There was no significant difference due to the selection of kernel - RBF or Linear. From the results, it could be suggested that it was possible to predict the type of task that a user was performing based on the short analysis of the typing behavior of that user. The results of the binary classification showed some minor differences between the low cognitive demand versus the high cognitive demand.
- Prediction of demography: Several experiments were performed to predict handedness, gender, and native language. They extracted the demographic labels based on self-reports of the users. The experiments were performed on training and test data. They used four classifiers for their experiments: LogicBoost, NB, SMO (RBFKernel), and SimpleLogistic. The Simple Logistic classifier performed better than other classifiers for gender classification. The NB classifier generated the most promising results for native language and handedness prediction. The demographic prediction results did not vary due to cognitive tasks. Prediction results were computed for each answer. They were able to accurately predict all 3 demographics for 55% of the answers. A Minimum of 2 of the 3 demographics were correctly predicted for 95% of the answers. Among the features, three features were very helpful for gender classification: (1) the timing between punctuation and hitting the spacebar, (2) the timing before and after function keys, and (3) the timing before and after common digraphs. It was observed that non-native English speakers typed more slowly than native English speakers. Mainly, three features were important for primary language prediction: (1) the timing before and after function keys, (2) the timing before and after the ‘.’ key, and (3) the timing before and after common digraphs.

7.4 Disease Detection

It is desirable to use keystroke dynamics for self-reporting, and self-monitoring for medical conditions, to include capturing indicators of declining health. The purpose is to identify and detect changes in typing behavior believed to be the same user, but changes in typing patterns suggest testing for health risks. Parkinson’s Disease is a highly misdiagnosed, progressive neurodegenerative movement disease that exhibits a change in motor function over time, which can be measured in the fingers. Research has been conducted for the early detection of Parkinson’s Disease by detecting changes in the characteristics of finger movements typed on a keyboard to

classify people with Parkinson's from those without the disease. Patients without Parkinson's disease were distinguished from those who were diagnosed [4]. It was also theorized that a simpler model to classify keystroke variances can be effective while reducing computing resources and requiring fewer keystrokes for training, making such detection systems practicable [71].

7.5 Mental Fatigue Detection

Conducting intricate tasks requires attentiveness. Fatigue contributes to errors and mistakes. Long-haul truck drivers who have worked long hours often suffer from an increase in fatigue-related accidents. Drivers are now required to comply with Electronic Logging Devices, or ELD, to limit the time on the road in an attempt to reduce accidents. Electronic logbooks are congressionally mandated and intended to help create a safer work environment for drivers. An ELD synchronizes with a vehicle engine to automatically record driving time for easier and more accurate hours of service recording. Similar to driving risks, other professions, such as surgeons, first responders, and pilots, can also benefit from fatigue detection. Instead of limiting productivity by time, dangerous levels of fatigue can be detected by the way a user types using Keystroke Dynamics [3].

8 Research Opportunities

The current landscape of keystroke dynamics authentication systems presents several limitations and challenges that need to be addressed to enhance security and user experience. Firstly, some conservative users may find keystroke dynamics authentication inconvenient, which may lead to reluctance in its adoption. Additionally, these systems are vulnerable to spoofing attacks in which synthetic keystroke dynamics samples can be forged and maliciously used to compromise security. Furthermore, various factors such as noise, different keyboards, typing on unfamiliar devices, physical condition, or urgency can affect a user's typing pattern or speed. Therefore, these factors need to be considered seriously when designing keystroke dynamics-based systems. Moreover, ensuring the privacy and security of users' keystroke data is paramount.

To overcome these challenges and improve keystroke dynamics-based systems, several research directions can be explored.

- **Research Benchmarks:** Shared research benchmarks are necessary to truly understand the strengths and limitations of each algorithm and compare them fairly and reliably. For example, Stragapede et al. present a new experimental framework to benchmark keystroke dynamics-based biometric verification performance and fairness [102].
- **Adaptive Authentication Systems:** Researchers can focus on developing novel keystroke authentication systems that can *adapt* to the changing typing behaviors of users in various circumstances. This adaptability can significantly improve authentication performance by accommodating variations in typing patterns.
- **Optimization of Training Procedures:** In order to optimize the training procedure of keystroke authentication models, an embedding vector model can be learned as commonly done in the computer vision field. A much larger dataset of keystroke sequences would need to be collected. The model will be trained using keystroke sequences as input. The learned model will convert each keystroke sequence or feature set into an embedding vector representation. For this procedure, neural network architectures can be adopted, such as RNN, CNN, and transformer.
- **Addressing Synthetic Forgeries:** Synthetic forgeries of keystroke dynamics samples are a potential research area. Gonzalez [43] introduced various innovative techniques to generate counterfeit keystroke dynamics samples using genuine free-text samples provided by legitimate users. In addition, a liveness detection approach was proposed that could

differentiate between human-written samples and synthetic forgeries. Gonzalez et al. [44] presented two approaches to create synthetic keystroke timing samples, as well as a live-ness detection system for keystroke dynamics that employed these synthetic samples as adversaries.

- **Employment of Cryptographic Techniques:** To ensure the privacy and security of users' keystroke data, cryptographic techniques can be employed. These techniques can include encryption, hashing, and secure transmission protocols to safeguard sensitive keystroke data from unauthorized access or tampering.
- **Integration with Other Biometric Modalities:** To increase the robustness of authentication, keystroke dynamics can be combined with other biometric modalities such as face or fingerprint recognition. This multi-modal approach would enhance security and accuracy by adding additional layers of authentication.

By addressing these research opportunities, keystroke dynamics-based authentication systems can become more robust, secure, and user-friendly, thus, fostering greater trust and adoption among users.

9 Conclusion

Keystroke dynamics is a very effective form of behavioral biometrics and remains an area of active research. This survey reviews various aspects of keystroke dynamics, which can be very helpful for future research in this domain. Keystroke dynamics can be a very powerful tool for security and authentication, a significant reason for which is that the application and deployment of keystroke dynamics-based systems is cost-effective and user-friendly. This survey covers the state-of-the-art of keystroke dynamics research and applications. This includes the most important keystroke datasets as well as keystroke authentication algorithms. Different keystroke feature engineering approaches like text filtering, data size variation, removing faulty data, and using non-conventional keystroke features are described. It also covers aspects of keystroke authentication on touch screen and mobile devices. Finally, various applications of keystroke dynamics are described in detail.

References

- [1] Alejandro Acien, Aythami Morales, John V. Monaco, Ruben Vera-Rodriguez, and Julian Fierrez. 2021. TypeNet: Deep learning keystroke biometrics. *IEEE Transactions on Biometrics, Behavior, and Identity Science* 4, 1 (2021), 57–70.
- [2] Alejandro Acien, Aythami Morales, Ruben Vera-Rodriguez, and Julian Fierrez. 2019. Keystroke mobile authentication: Performance of long-term approaches and fusion with behavioral profiling. In *Proceedings of the Iberian Conference on Pattern Recognition and Image Analysis*. Springer, 12–24.
- [3] Alejandro Acien, Aythami Morales, Ruben Vera-Rodriguez, Julian Fierrez, Ijah Mondesire-Crump, and Teresa Arroyo-Gallego. 2022. Detection of mental fatigue in the general population: Feasibility study of keystroke dynamics as a real-world biomarker. *JMIR Biomedical Engineering* 7, 2 (2022), e41003.
- [4] Warwick R. Adams. 2017. High-accuracy detection of early Parkinson's disease using multiple characteristics of finger movement while typing. *PloS One* 12, 11 (2017), e0188226.
- [5] Ahmed A. Ahmed and Issa Traore. 2013. Biometric recognition based on free-text keystroke dynamics. *IEEE Transactions on Cybernetics* 44, 4 (2013), 458–472.
- [6] Noor Mahmood Al-Obaidi and Mudhafar M. Al-Jarrah. 2016. Statistical keystroke dynamics system on mobile devices for experimental data collection and user authentication. In *Proceedings of the 2016 9th International Conference on Developments in eSystems Engineering (DeSE)*. IEEE, 123–129.
- [7] Noor Mahmood Al-Obaidi and Mudhafar M. Al-Jarrah. 2016. Statistical median-based classifier model for keystroke dynamics on mobile devices. In *Proceedings of the 2016 6th International Conference on Digital Information Processing and Communications (ICDIPC)*. IEEE, 186–191.
- [8] Md Liakat Ali, John V. Monaco, Charles C. Tappert, and Meikang Qiu. 2017. Keystroke biometric systems for user authentication. *Journal of Signal Processing Systems* 86, 2 (2017), 175–190.

- [9] Md Liakat Ali, Charles C. Tappert, Meikang Qiu, and John V. Monaco. 2015. Authentication and identification methods used in keystroke biometric systems. In *Proceedings of the 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems*. IEEE, 1424–1429.
- [10] Faisal Alshanketi, Issa Traore, and Ahmed Awad Ahmed. 2016. Improving performance and usability in mobile keystroke dynamic biometric authentication. In *Proceedings of the 2016 IEEE Security and Privacy Workshops (SPW)*. IEEE, 66–73.
- [11] Arwa Alsultan and Kevin Warwick. 2013. Keystroke dynamics authentication: A survey of free-text methods. *International Journal of Computer Science Issues* 10, 4 (2013), 1.
- [12] Arwa Alsultan, Kevin Warwick, and Hong Wei. 2017. Non-conventional keystroke dynamics for user authentication. *Pattern Recognition Letters* 89, 4 (2017), 53–59.
- [13] Margit Antal and László Zsolt Szabó. 2015. An evaluation of one-class and two-class classification algorithms for keystroke dynamics authentication on mobile devices. In *Proceedings of the 2015 20th International Conference on Control Systems and Computer Science*. IEEE, 343–350.
- [14] Margit Antal, László Zsolt Szabó, and Izabella László. 2015. Keystroke dynamics on android platform. *Procedia Technology* 19 (2015), 820–826.
- [15] Tanapat Anusas-Amornkul. 2019. Strengthening password authentication using keystroke dynamics and smart-phone sensors. In *Proceedings of the 9th International Conference on Information Communication and Management*. 70–74.
- [16] Blaine Ayotte, Mahesh Banavar, Daqing Hou, and Stephanie Schuckers. 2020. Fast free-text authentication via instance-based keystroke dynamics. *IEEE Transactions on Biometrics, Behavior, and Identity Science* 2, 4 (2020), 377–387.
- [17] Ned Bakelman, John V. Monaco, Sung-Hyuk Cha, and Charles C. Tappert. 2013. Keystroke biometric studies on password and numeric keypad input. In *Proceedings of the 2013 European Intelligence and Security Informatics Conference*. IEEE, 204–207.
- [18] Paul E. Black. 2019. Manhattan distance. Retrieved June 15, 2019 from <https://www.nist.gov/dads/HTML/manhattanDistance.html>
- [19] David Guy Brizan, Adam Goodkind, Patrick Koch, Kiran Balagani, Vir V. Phoha, and Andrew Rosenberg. 2015. Utilizing linguistically enhanced keystroke dynamics to predict typist cognition and demographics. *International Journal of Human-Computer Studies* 82, 10 (2015), 57–68.
- [20] D. Broomhead and D. Lowe. 1988. Multivariable functional interpolation and adaptive networks. *Complex Systems* 2, 3 (1988), 321–355.
- [21] Hayreddin Çeker and Shambhu Upadhyaya. 2015. Enhanced recognition of keystroke dynamics using Gaussian mixture models. In *Proceedings of the MILCOM 2015-2015 IEEE Military Communications Conference*. IEEE, 1305–1310.
- [22] Hayreddin Çeker and Shambhu Upadhyaya. 2016. User authentication with keystroke dynamics in long-text data. In *Proceedings of the 2016 IEEE 8th International Conference on Biometrics Theory, Applications and Systems (BTAS)*. IEEE, 1–6.
- [23] Hayreddin Çeker and Shambhu Upadhyaya. 2017. Sensitivity analysis in keystroke dynamics using convolutional neural networks. In *Proceedings of the 2017 IEEE Workshop on Information Forensics and Security (WIFS)*. IEEE, 1–6.
- [24] Hayreddin Çeker and Shambhu Upadhyaya. 2017. Transfer learning in long-text keystroke dynamics. In *Proceedings of the 2017 IEEE International Conference on Identity, Security, and Behavior Analysis (ISBA)*. IEEE, 1–6.
- [25] Han-Chih Chang, Jianwei Li, Ching-Seh Wu, and Mark Stamp. 2022. Machine learning and deep learning for fixed-text keystroke dynamics. In *Proceedings of the Cybersecurity for Artificial Intelligence*. Springer, 309–329.
- [26] Zhaohang Chen, Hongming Cai, Lihong Jiang, WenYun Zou, Wendong Zhu, and Xiang Fei. 2021. Keystroke dynamics based user authentication and its application in online examination. In *Proceedings of the 2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 649–654.
- [27] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar. Association for Computational Linguistics, 1724–1734.
- [28] Alaa Darabseh and Akbar Siami Namin. 2015. Effective user authentications using keystroke dynamics based on feature selections. In *Proceedings of the 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 307–312.
- [29] Debayan Deb, Arun Ross, Anil K. Jain, Kwaku Prakah-Asante, and K. Venkatesh Prasad. 2019. Actions speak louder than (pass) words: Passive authentication of smartphone* users via deep temporal features. In *Proceedings of the 2019 International Conference on Biometrics (ICB)*. IEEE, 1–8.

- [30] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. 1977. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)* 39, 1 (1977), 1–22.
- [31] Yunbin Deng and Yu Zhong. 2013. Keystroke dynamics user authentication based on Gaussian mixture model and deep belief nets. *International Scholarly Research Notices* 2013, 1 (2013), 565183.
- [32] Vivek Dhakal, Anna Maria Feit, Per Ola Kristensson, and Antti Oulasvirta. 2018. Observations on typing from 136 million keystrokes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [33] Aleix Dorca Josa. 2017. *Identifying Users Using Keystroke Dynamics and Contextual Information*. Ph.D. Dissertation. Universitat d'Andorra.
- [34] Mohamad El-Abed, Mostafa Dafer, and Ramzi El Khayat. 2014. RHU keystroke: A mobile-based benchmark for keystroke dynamics systems. In *Proceedings of the 2014 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 1–4.
- [35] Andrew Foresi and Reza Samavi. 2019. User authentication using keystroke dynamics via crowdsourcing. In *Proceedings of the 2019 17th International Conference on Privacy, Security and Trust (PST)*. IEEE, 1–3.
- [36] Kunihiro Fukushima and Sei Miyake. 1982. Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. In *Proceedings of the Competition and Cooperation in Neural Nets*. Springer, 267–285.
- [37] Ashish Garg, Shambhu Upadhyaya, and Kevin Kwiat. 2013. A user behavior monitoring and profiling scheme for masquerade detection. In *Proceedings of the Handbook of Statistics*. Elsevier, 353–379.
- [38] Romain Giot, Bernadette Dorizzi, and Christophe Rosenberger. 2015. A review on the public benchmark databases for static keystroke dynamics. *Computers and Security* 55, 11 (2015), 46–61.
- [39] Romain Giot, Mohamad El-Abed, Baptiste Hemery, and Christophe Rosenberger. 2011. Unconstrained keystroke dynamics authentication with shared secret. *Computers and Security* 30, 6–7 (2011), 427–445.
- [40] Romain Giot, Mohamad El-Abed, and Christophe Rosenberger. 2009. Greyc keystroke: A benchmark for keystroke dynamics biometric systems. In *Proceedings of the 2009 IEEE 3rd International Conference on Biometrics: Theory, Applications, and Systems*. IEEE, 1–6.
- [41] Romain Giot, Mohamad El-Abed, and Christophe Rosenberger. 2009. Keystroke dynamics with low constraints svm based passphrase enrollment. In *Proceedings of the 2009 IEEE 3rd International Conference on Biometrics: Theory, Applications, and Systems*. IEEE, 1–6.
- [42] Romain Giot, Mohamad El-Abed, and Christophe Rosenberger. 2012. Web-based benchmark for keystroke dynamics biometric systems: A statistical analysis. In *Proceedings of the 2012 8th International Conference on Intelligent Information Hiding and Multimedia Signal Processing*. IEEE, 11–15.
- [43] Nahuel González. 2023. Ksdslid—a tool for keystroke dynamics synthesis and liveness detection. *Software Impacts* 15, 3 (2023), 100454.
- [44] Nahuel González, Enrique P. Calot, Jorge S. Ierache, and Waldo Hasperué. 2022. Towards liveness detection in keystroke dynamics: Revealing synthetic forgeries. *Systems and Soft Computing* 4, 12 (2022), 200037.
- [45] Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Shuai, Ting Liu, Xingxing Wang, Gang Wang, Jianfei Cai, et al. 2018. Recent advances in convolutional neural networks. *Pattern Recognition* 77, 5 (2018), 354–377.
- [46] Daniele Gunetti and Claudia Picardi. 2005. Keystroke analysis of free text. *ACM Transactions on Information and System Security* 8, 3 (2005), 312–347.
- [47] Norhaslinda Harun, Satnam Singh Dlay, and Wai Lok Woo. 2010. Performance of keystroke biometrics authentication system using multilayer perceptron neural network (MLP NN). In *Proceedings of the 2010 7th International Symposium on Communication Systems, Networks and Digital Signal Processing (CSNDSP 2010)*. IEEE, 711–714.
- [48] Trevor Hastie and Robert Tibshirani. 1997. Classification by pairwise coupling. *Advances in Neural Information Processing Systems* 10 (1997), 507–513.
- [49] Jiacang Ho and Dae-Ki Kang. 2018. One-class naïve bayes with duration feature ranking for accurate user authentication using keystroke dynamics. *Applied Intelligence* 48, 6 (2018), 1547–1564.
- [50] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.
- [51] Jiankun Hu, Don Gingrich, and Andy Sentosa. 2008. A k-nearest neighbor approach for user authentication through biometric keystroke dynamics. In *Proceedings of the 2008 IEEE International Conference on Communications*. IEEE, 1556–1560.
- [52] Jiaju Huang, Daqing Hou, Stephanie Schuckers, and Zhenhao Hou. 2015. Effect of data size on performance of free-text keystroke authentication. In *Proceedings of the IEEE International Conference on Identity, Security and Behavior Analysis (ISBA 2015)*. IEEE, 1–7.
- [53] Jiaju Huang, Daqing Hou, Stephanie Schuckers, Timothy Law, and Adam Sherwin. 2017. Benchmarking keystroke authentication algorithms. In *Proceedings of the 2017 IEEE Workshop on Information Forensics and Security (WIFS)*. IEEE, 1–6.

- [54] Jiaju Huang, Daqing Hou, Stephanie Schuckers, and Shambhu Upadhyaya. 2016. Effects of text filtering on authentication performance of keystroke biometrics. In *Proceedings of the 2016 IEEE International Workshop on Information Forensics and Security (WIFS)*. IEEE, 1–6.
- [55] Elena Ivannikova, Gil David, and Timo Hääläinen. 2017. Anomaly detection approach to keystroke dynamics based user authentication. In *Proceedings of the 2017 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 885–889.
- [56] Kevin Killourhy and Roy Maxion. 2009. Keystroke dynamics-benchmark dataset. *Carnegie-Mellon University*, <http://www.cs.cmu.edu/~keystroke/#sec2> (2009).
- [57] Kevin Killourhy and Roy Maxion. 2010. Why did my detector do that?!. In *Proceedings of the International Workshop on Recent Advances in Intrusion Detection*. Springer, 256–276.
- [58] Kevin S. Killourhy and Roy A. Maxion. 2009. Comparing anomaly-detection algorithms for keystroke dynamics. In *Proceedings of the 2009 IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 125–134.
- [59] Junhong Kim and Pilsung Kang. 2020. Freely typed keystroke dynamics-based user authentication for mobile devices based on heterogeneous features. *Pattern Recognition* 108, 12 (2020), 107556.
- [60] Paweł Kobjek and Khalid Saeed. 2016. Application of recurrent neural networks for user verification based on keystroke dynamics. *Journal of Telecommunications and Information Technology* 3, 3 (2016), 80–90.
- [61] Sowndarya Krishnamoorthy, Luis Rueda, Sherif Saad, and Haytham Elmiligi. 2018. Identification of user behavioral biometrics for authentication using keystroke dynamics and machine learning. In *Proceedings of the 2018 2nd International Conference on Biometric Engineering and Applications*. 50–57.
- [62] John Leggett and Glen Williams. 1988. Verifying identity via keystroke characteristics. *International Journal of Man-Machine Studies* 28, 1 (1988), 67–76.
- [63] John Leggett, Glen Williams, Mark Usnick, and Mike Longnecker. 1991. Dynamic identity verification via keystroke characteristics. *International Journal of Man-Machine Studies* 35, 6 (1991), 859–870.
- [64] Jianwei Li, Han-Chih Chang, and Mark Stamp. 2022. Free-text keystroke dynamics for user authentication. In *Proceedings of the Cybersecurity for Artificial Intelligence*. Springer, 357–380.
- [65] Chu-Hsing Lin, Jung-Chun Liu, and Ken-Yu Lee. 2018. On neural networks for biometric authentication based on keystroke dynamics. *Sensors and Materials* 30, 3 (2018), 385–396.
- [66] Gordon D. Logan, Jana E. Ulrich, and Dakota R. B. Lindsey. 2016. Different (key) strokes for different folks: How standard and nonstandard typists balance Fitts’ law and Hick’s law. *Journal of Experimental Psychology: Human Perception and Performance* 42, 12 (2016), 2084.
- [67] Xiaofeng Lu, Shengfei Zhang, Pan Hui, and Pietro Lio. 2020. Continuous authentication by free-text keystroke based on CNN and RNN. *Computers and Security* 96, 9 (2020), 101861.
- [68] Emanuele Maiorana, Himanka Kalita, and Patrizio Campisi. 2021. Mobile keystroke dynamics for biometric recognition: An overview. *IET Biometrics* 10, 1 (2021), 1–23.
- [69] Danilo Mandic and Jonathon Chambers. 2001. *Recurrent Neural Networks for Prediction: Learning Algorithms, Architectures and Stability*. Wiley.
- [70] Denis Migdal and Christophe Rosenberger. 2018. Analysis of keystroke dynamics for the generation of synthetic datasets. In *Proceedings of the 2018 International Conference on Cyberworlds (CW)*. IEEE, 339–344.
- [71] Antony Milne, Katayoun Farrahi, and Mihalīs A. Nicolaou. 2018. Less is more: Univariate modelling to detect early Parkinson’s disease from keystroke dynamics. In *Proceedings of the International Conference on Discovery Science*. Springer, 435–446.
- [72] Martha Mohlala, Adeyemi R. Ikuesan, and Hein S. Venter. 2017. User attribution based on keystroke dynamics in digital forensic readiness process. In *Proceedings of the 2017 IEEE Conference on Application, Information and Network Security (AINS)*. IEEE, 124–129.
- [73] John V. Monaco, Gonzalo Perez, Charles C. Tappert, Patrick Bours, Soumik Mondal, Sudalai Rajkumar, Aythami Morales, Julian Fierrez, and Javier Ortega-Garcia. 2015. One-handed keystroke biometric identification competition. In *Proceedings of the 2015 International Conference on Biometrics (ICB)*. IEEE, 58–64.
- [74] Soumik Mondal and Patrick Bours. 2017. Person identification by keystroke dynamics using pairwise user coupling. *IEEE Transactions on Information Forensics and Security* 12, 6 (2017), 1319–1329.
- [75] Soumik Mondal and Patrick Bours. 2017. A study on continuous authentication using a combination of keystroke and mouse biometrics. *Neurocomputing* 230, 3 (2017), 1–22.
- [76] Christopher Murphy, Jiaju Huang, Daqing Hou, and Stephanie Schuckers. 2017. Shared dataset on natural human-computer interaction to support continuous authentication research. In *Proceedings of the 2017 IEEE International Joint Conference on Biometrics (IJCB)*. IEEE, 525–530.
- [77] AFM Nazmul Haque Nahin, Jawad Mohammad Alam, Hasan Mahmud, and Kamrul Hasan. 2014. Identifying emotion by keystroke dynamics and text pattern analysis. *Behaviour and Information Technology* 33, 9 (2014), 987–996.

- [78] Teodor Neacsu, Teodor Poncu, Stefan Ruseti, and Mihai Dascalu. 2023. DoubleStrokeNet: Bigram-level keystroke authentication. *Electronics* 12, 20 (2023), 4309.
- [79] Mehmet Erdal ÖZBEK. 2019. Classification performance improvement of keystroke data. In *Proceedings of the 2019 Innovations in Intelligent Systems and Applications Conference (ASYU)*. IEEE, 1–4.
- [80] Kseniia Palin, Anna Maria Feit, Sunjun Kim, Per Ola Kristensson, and Antti Oulasvirta. 2019. How do people type on mobile devices? Observations from a study with 37,000 volunteers. In *Proceedings of the 21st International Conference on Human-Computer Interaction with Mobile Devices and Services*. 1–12.
- [81] Avar Pentel. 2017. Predicting age and gender by keystroke dynamics and mouse patterns. In *Proceedings of the Adjunct Publication of the 25th Conference on User Modeling, Adaptation and Personalization*. 381–385.
- [82] Eduard C. Popovici, Liviu A. Stancu, Ovidiu G. Guta, Stefan C. Arseni, and Octavian Fratu. 2014. Combined use of pattern recognition algorithms for keystroke-based continuous authentication system. In *Proceedings of the 2014 10th International Conference on Communications (COMM)*. IEEE, 1–4.
- [83] Yuqing Qi, Weichen Jia, and Shuo Gao. 2021. Emotion recognition based on piezoelectric keystroke dynamics and machine learning. In *Proceedings of the 2021 IEEE International Conference on Flexible and Printable Sensors and Systems (FLEPS)*. IEEE, 1–4.
- [84] Sriram Ravindran, Chandan Gautam, and Aruna Tiwari. 2015. Keystroke user recognition through extreme learning machine and evolving cluster method. In *Proceedings of the 2015 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*. IEEE, 1–5.
- [85] Siti Fairuz Nurr Sadikan, Azizul Azhar Ramli, and Mohd Farhan Md Fudzee. 2019. A survey paper on keystroke dynamics authentication for current applications. *AIP Conference Proceedings*, Vol. 2173. AIP Publishing LLC.
- [86] Napa Sae-Bae and Nasir Memon. 2022. Distinguishability of keystroke dynamic template. *PloS One* 17, 1 (2022), e0261291.
- [87] Ramzi Saifan, Asma Salem, Dema Zaidan, Andraws Swidan, et al. 2016. A survey of behavioral authentication using keystroke dynamics: Touch screens and mobile devices. *Journal of Social Sciences* 5, 1 (2016), 29–41.
- [88] Baljit Singh Saini, Navdeep Kaur, and Kamaljit Singh Bhatia. 2016. Keystroke dynamics for mobile phones: A survey. *Indian Journal of Science and Technology* 9, 6 (2016), 1–8.
- [89] Gerard Salton, Anita Wong, and Chung-Shu Yang. 1975. A vector space model for automatic indexing. *Communications of the ACM* 18, 11 (1975), 613–620.
- [90] Akash Sanghi and Y. D. S. Arya. 2017. Survey, applications and security of keystroke dynamics for user authentication. *International Journal of Computer and Mathematical Sciences (IJCMS)*, ISSN 9, 6 (2017), 2347–8527.
- [91] K. R. Scherer and H. Wallbott. 1994. Evidence for universality and cultural variation of differential emotion response patterning. *Journal of Personality and Social Psychology* 66, 2 (1994), 310–328. <https://doi.org/10.1037/0022-3514.66.2.310>
- [92] Klaus R. Scherer and Harald G. Wallbott. 1994. Evidence for universality and cultural variation of differential emotion response patterning. *Journal of Personality and Social Psychology* 66, 2 (1994), 310.
- [93] Chao Shen, Zhongmin Cai, Xiaohong Guan, Youtian Du, and Roy A. Maxion. 2012. User authentication through mouse dynamics. *IEEE Transactions on Information Forensics and Security* 8, 1 (2012), 16–30.
- [94] Yong Sheng, Vir V. Phoha, and Steven M. Rovnyak. 2005. A parallel decision tree-based method for user authentication based on keystroke patterns. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 35, 4 (2005), 826–833.
- [95] Pranit Shinde, Saideep Shetty, and Mahendra Mehra. 2016. Survey of keystroke dynamics as a biometric for static authentication. *International Journal of Computer Science and Information Security* 14, 4 (2016), 203.
- [96] Saurabh Singh. 2018. Keystroke dynamics for continuous authentication. In *Proceedings of the 2018 8th International Conference on Cloud Computing, Data Science and Engineering (Confluence)*. IEEE, 205–208.
- [97] Shivshakti Singh, Aditi Inamdar, Aishwarya Kore, and Aprupa Pawar. 2020. Analysis of algorithms for user authentication using keystroke dynamics. In *Proceedings of the 2020 International Conference on Communication and Signal Processing (ICCSP)*. IEEE, 0337–0341.
- [98] Mahalaxmi Sridhar, Siddhesh Vaidya, and Piyush Yawalkar. 2015. Intrusion detection using keystroke dynamics and fuzzy logic membership functions. In *Proceedings of the 2015 International Conference on Technologies for Sustainable Development (ICTSD)*. IEEE, 1–10.
- [99] Valeriu-Daniel Stanciu, Riccardo Spolaor, Mauro Conti, and Cristiano Giuffrida. 2016. On the effectiveness of sensor-enhanced keystroke dynamics against statistical attacks. In *Proceedings of the 6th ACM Conference on Data and Application Security and Privacy*. 105–112.
- [100] Giuseppe Stragapede, Paula Delgado-Santos, Ruben Tolosana, Ruben Vera-Rodriguez, Richard Guest, and Aythami Morales. 2024. TypeFormer: Transformers for mobile keystroke biometrics. *Neural Computing and Applications* 36, 29 (2024), 18531–18545.

- [101] Giuseppe Stragapede, Ruben Vera-Rodriguez, Ruben Tolosana, and Aythami Morales. 2023. BehavePassDB: Public database for mobile behavioral biometrics and benchmark evaluation. *Pattern Recognition* 134, 2 (2023), 109089.
- [102] Giuseppe Stragapede, Ruben Vera-Rodriguez, Ruben Tolosana, Aythami Morales, Naser Damer, Julian Fierrez, and Javier Ortega-Garcia. 2023. Keystroke verification challenge (KVC): Biometric and fairness benchmark evaluation. *IEEE Access* 12, 12 (2023), 1102–1116.
- [103] Ioannis Stylios, Andreas Skalkos, Spyros Kokolakis, and Maria Karyda. 2021. Bioprivacy: Development of a keystroke dynamics continuous authentication system. In *Proceedings of the European Symposium on Research in Computer Security*. Springer, 158–170.
- [104] Yan Sun, Hayreddin Ceker, and Shambhu Upadhyaya. 2016. Shared keystroke dataset for continuous authentication. In *Proceedings of the 2016 IEEE International Workshop on Information Forensics and Security (WIFS)*. IEEE, 1–6.
- [105] Yan Sun and Shambhu Upadhyaya. 2015. Secure and privacy preserving data processing support for active authentication. *Information Systems Frontiers* 17, 5 (2015), 1007–1015.
- [106] Zahid Syed, Sean Banerjee, and Bojan Cukic. 2014. Leveraging variations in event sequences in keystroke-dynamics authentication systems. In *Proceedings of the 2014 IEEE 15th International Symposium on High-Assurance Systems Engineering*. IEEE, 9–16.
- [107] Pin Shen Teh, Andrew Beng Jin Teoh, and Shigang Yue. 2013. A survey of keystroke dynamics biometrics. *The Scientific World Journal* 2013, 1 (2013), 408280.
- [108] Anurag Tewari. 2022. Keystroke dynamics based recognition systems using deep learning: A survey. Authorea Preprints.
- [109] Chee Meng Tey, Payas Gupta, and Debin Gao. 2013. I can be you: Questioning the use of keystroke dynamics as biometrics. *Annual Network and Distributed System Security Symposium 20th NDSS 2013*. 1–16.
- [110] Ioannis Tsimperidis, Avi Arampatzis, and Alexandros Karakos. 2018. Keystroke dynamics features for gender recognition. *Digital Investigation* 24, 3 (2018), 4–10.
- [111] Ioannis Tsimperidis, Paul D. Yoo, Kamal Taha, Alexios Mylonas, and Vasilis Katos. 2018. R 2 BN: An adaptive model for keystroke-dynamics-based educational level classification. *IEEE Transactions on Cybernetics* 50, 2 (2018), 525–535.
- [112] Esra Vural, Jiaju Huang, Daqing Hou, and Stephanie Schuckers. 2014. Shared research dataset to support development of keystroke authentication. In *Proceedings of the IEEE International Joint Conference on Biometrics*. IEEE, 1–8.
- [113] Ahmed Anu Wahab and Daqing Hou. 2023. A user study of keystroke dynamics as second factor in web MFA. In *Proceedings of the 13th ACM Conference on Data and Application Security and Privacy*. 236–241.
- [114] Ahmed Anu Wahab, Daqing Hou, Mahesh Banavar, Stephanie Schuckers, Kenneth Eaton, Jacob Baldwin, and Robert Wright. 2022. Shared multi-keyboard and bilingual datasets to support keystroke dynamics research. In *Proceedings of the 12th ACM Conference on Data and Application Security and Privacy*. 236–241.
- [115] Ahmed Anu Wahab, Daqing Hou, Stephanie Schuckers, and Abbie Barbir. 2021. Utilizing keystroke dynamics as additional security measure to protect account recovery mechanism. In *Proceedings of the ICISSP*. 33–42.
- [116] Xiaofei Wang, Rashik Shadman, Daqing Hou, Faraz Hussain, and Stephanie Schuckers. 2024. A novel keystroke dataset for preventing advanced persistent threats. In *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods—ICPRAM*. INSTICC, SciTePress, 894–901. DOI: [10.5220/0012558000003654](https://doi.org/10.5220/0012558000003654)
- [117] Ian H. Witten, Eibe Frank, Mark A. Hall, Christopher J. Pal, and MINING DATA. 2005. Practical machine learning tools and techniques. In *Proceedings of the DATA MINING*. 4.
- [118] Changsheng Wu, Wenbo Ding, Ruiyuan Liu, Jiyu Wang, Aurelia C. Wang, Jie Wang, Shengming Li, Yunlong Zi, and Zhong Lin Wang. 2018. Keystroke dynamics enabled authentication and identification using triboelectric nanogenerator array. *Materials Today* 21, 3 (2018), 216–222.
- [119] Lu Xiaofeng, Zhang Shengfei, and Yi Shengwei. 2019. Continuous authentication by free-text keystroke based on CNN plus RNN. *Procedia Computer Science* 147, 1 (2019), 314–318.
- [120] Nan Zheng, Aaron Paloski, and Haining Wang. 2011. An efficient user verification system via mouse movements. In *Proceedings of the 18th ACM Conference on Computer and Communications Security*. 139–150.
- [121] Yu Zhong and Yunbin Deng. 2015. A survey on keystroke dynamics biometrics: Approaches, advances, and evaluations. *Recent Advances in User Authentication Using Keystroke Dynamics Biometrics* 1 (2015), 1–22.
- [122] Yu Zhong, Yunbin Deng, and Anil K. Jain. 2012. Keystroke dynamics for user authentication. In *Proceedings of the 2012 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, 117–123.

Received 9 April 2023; revised 25 June 2024; accepted 1 April 2025